VeriQKD-Net: Measurement-Driven Verification and Implementation Testing of QKD Key Delivery

ANONYMOUS AUTHOR(S)

Quantum key distribution (QKD) depends on classical software that selects, confirms, and consumes keys. We study this delivery boundary using public hardware data, bounded models, and executable storage code. IrelandQCI v2 contains 54,531 rows across five platforms. Excluding an exact 6,024-row auxiliary subset leaves 48,507 primary observations. Explicit accumulation assumptions produce 582,084 capacity mappings and seventeen resource profiles. Exhaustive checking covers 306 cases: atomic and reserved modes satisfy the contracts in all 204 controls; split delivery fails in 61 of 102 cases. A separately authored TLA+ specification checked by TLC agrees on all aggregate safety verdicts and all 245 complete passing state counts. A matched testing baseline executes 306,000 episodes and misses one failing case across the selected seeds. A newly implemented SQLite controller runs 2,040 concurrent batches covering 11,760 requests. It reproduces eighty stored witnesses with 430 matching count projections. It also exposes identifier reuse without excess capacity, which that projection cannot establish. Atomic and reserved implementations avoid the selected findings. A separate 522-configuration synthetic benchmark supplies fault sensitivity, 120,000 earlier traces, and repeated resource measurements. The contribution is a reproducible connection between measured supply assumptions, delivery contracts, diverse checking encodings, and implementation observations. Hardware rates are measured; workloads and timing budgets are modeled. The evidence does not prove full implementation refinement or deployed QKD reliability.

CCS Concepts: • **Software and its engineering → Software verification**; *Software testing and debugging*.

Additional Key Words and Phrases: QKD key management, model checking, implementation testing, concurrency, public hardware data

1 Introduction

A quantum key distribution (QKD) network is operated through classical software. Applications request keys. Controllers select service paths. Link modules report processing results. Relays and endpoint managers acknowledge a session. Finally, a key manager releases key material and changes its lifecycle state. The security contribution of a quantum link therefore depends on software that uses the resulting material correctly. QKD network key management is already a substantial engineering concern [1]. Delivery interfaces and network-management procedures are addressed by ETSI and ITU-T standards [2, 3].

Consider a key manager with a simple implementation: check that a key is unused, return it, then mark it consumed. If these operations are separate, two sessions may both observe the unused value before either update completes. A second problem occurs when a key expires after a successful check but before delivery. These are classical coordination faults. A correct underlying QKD primitive does not remove them. Likewise, a correct delivery check does not ensure that a retry loop terminates or that an abort has an authorized cause.

Formal specification and model checking provide established ways to examine such design decisions [4, 5]. However, a small successful example offers limited experimental evidence. One must also inspect faults, budgets, concurrent requests, and the assumptions that make a property pass. A delivery implication may pass only because no delivery is reachable. A faulty transition may be masked by a short event budget. A new checker may agree with its own exporter while sharing the same modeling error. These issues motivate a dataset with explicit configurations, unknown outcomes, counterexamples, and reproducible checking procedures.

2026. Manuscript submitted to ACM

This study combines public hardware measurements with an executable coordination study. IrelandQCI v2 supplies measured QBER and secure key rates for five commercial QKD platforms [6]. An audited subset of 48,507 records drives a transparent key-supply abstraction. The study also includes a structured synthetic benchmark for controlled coordination faults. VeriQKD-Bench contains 522 canonical configurations. It includes correct reference modes and deliberately unsafe alternatives. The experimental units are finite abstract models, not production deployments or independent real-world defects. The benchmark is designed to support inspection of coordination contracts and sensitivity to selected changes. It is not a benchmark of cryptographic algorithms, secret-key rates, or physical channel performance.

The central method separates several forms of evidence. A receipt records the preconditions at actual delivery. A persistent consumed flag records the required update afterwards. Intermediate predicates check processing order even before delivery. A cause snapshot checks whether abort was authorized when taken. A temporal obligation checks universal eventual termination on the totalized operational graph. This separation allows a trace to identify whether the failure concerns a precondition, an update, or progress.

The paper makes five contributions. First, it audits public hardware records and makes the assumptions linking secure key rates to finite key-unit capacities traceable. Second, it separates actual-delivery guards, lifecycle updates, and progress, allowing split selection and reservation to be compared under matched resource assumptions. Third, it links stored counterexamples to executable SQLite transactions and shows a concrete limit of a count projection: identifier reuse can occur without exceeding capacity. Fourth, it checks the resource safety verdicts against a separately authored TLA+ model executed by TLC and compares them with matched seeded execution testing. Fifth, it supplies the original synthetic coordination benchmark, raw experiment records, and replay audits as reproducible evidence.

The contribution is this domain-specific experimental connection between measured supply assumptions, delivery-boundary contracts, independent encodings, and implementation observations. It introduces no new temporal logic or checking algorithm. Check/commit races, reservations, systematic testing, and model-to-code validation are established ideas. The new evidence concerns how they interact in this QKD key-unit case study, including a capacity-versus-identity distinction that the resource abstraction alone cannot establish. Neither graph size nor the number of repeated tests is presented as methodological novelty.

Table 1 connects each question to a concrete record. The analysis distinguishes completed verification from resource-limited exploration. It also distinguishes a scheduler-dependent testing hit rate from an exhaustive property verdict. These distinctions are necessary for interpreting a large experiment without inflating what the data show.

2 Software Problem and Assumptions

2.1 Service boundary

The verification target is a classical coordination service. Link-level raw-key creation, error correction, and privacy amplification are represented by completion flags. No quantum state, key bit string, eavesdropper, detector, or channel-noise distribution is represented. A flag indicates that the relevant primitive has reported completion; it does not prove the primitive secure. The model is therefore not an implementation of BB84 or a cryptographic security proof.

A service session has a fixed application request and a finite route choice. Selecting a route creates one local record for each hop. Different hops may advance in different orders. Relays acknowledge only after all hop records have completed post-processing. Endpoint confirmation follows the required acknowledgements. Storage and delivery then complete

Table 1. Research questions and the evidence used to answer them.

RQ	Question	Evidence
RQ1	How do measured operating conditions map to verified key-supply cases?	Audited hardware rows, capacity crosswalk, resource-model checks.
RQ2	What completes, fails, or remains unknown across the design?	Frozen configurations, reachability, cap records.
RQ3	How do budgets, delays, and fault locations affect detection?	First-order and localized fault contexts.
RQ4	What do separate contract groups add?	Property-subset comparison on identical outcomes.
RQ5	Does split delivery expose a race, and does reservation repair it?	Matched delivery modes, shared/separate keys, traces.
RQ6	What does seeded execution testing miss?	Exhaustive scenario verdicts and 120,000 sampled traces.
RQ7	What are the observed time, memory, and state-space costs?	Fresh-process repeats and explicit unknown outcomes.
RQ8	How well do the evidence controls agree?	C++ graph cross-checking, witness replay, trace replay.
RQ9	Do separately encoded safety checks agree, and what does matched testing miss?	TLC checks of 306 cases; 306,000 additional executions.
RQ10	Which delivery faults appear in an executable storage controller?	SQLite transactions, 2,040 batches, and eighty witness replays.

the normal path. Error and expiry branches can terminate by abort. Key-supply protection and recovery are important operational topics [7]; the present model includes safe termination rather than a full failover or replenishment protocol.

Table 2. Explicit abstraction assumptions and their consequences.

Assumption	Consequence
Atomic transition execution	No interleaving occurs inside one model action.
Atomic reservation acquisition	Reserved-mode safety does not establish correctness of a non-atomic database implementation.
Fixed session identities	Session identity is represented by tuple position; acknowledgements are summarized by a mask and an identity-validity flag.
Local event budget	A deadline bounds model actions within one session, not elapsed physical time.
Trusted primitive reports	Correct reports are assumed; false device reports and cryptographic attacks are excluded.
Finite initial requests	No unbounded arrival process, repeated service loop, or key replenishment is modeled.
Trusted relays	Compromised relays and malicious controllers are outside the threat model.

Table 2 defines the conditions under which the results apply. In particular, a relay chain is represented by hop count and relay acknowledgement count. The artifact has no physical graph, routing optimizer, or shared-link bandwidth model. The alternative-route experiment selects between two route lengths; it is not a branching-network routing experiment.

2.2 Fault model

The catalog targets software omissions and ordering errors: starting link work without authentication, performing privacy amplification before error correction, confirming without complete acknowledgements, accepting an acknowledgement with invalid session identity, storing without endpoint confirmation, delivering without storage, delivering an expired key, omitting the consumed update, aborting without cause, losing timeout termination, and retrying without reducing a progress budget. A separate variant changes only an abort diagnostic label. It provides a control for distinguishing state-observable faults from label changes.

The catalog is constructed from the requirements and is not a sample of production bugs. Detection demonstrates sensitivity to those variants. It does not estimate the probability of finding real defects or the completeness of the property suite. No claim of exhaustive coverage of QKD software faults follows from a high score on this catalog.

3 Executable Reference Model

3.1 State representation

Let H_i denote the number of hops in the selected route for session i. The global state is

$$s = \langle q_1, \ldots, q_n, C, O \rangle,$$

where C is a Boolean vector recording consumed key identifiers. The extended implementation also contains a reservation-owner vector O, whose entries are a session index or -1 for unreserved. Each session record is

$$q_i = \langle \rho, a, \ell_1, \ldots, \ell_{H_i}, A, b, c, u, x, e,$$
$$t, z, R, v, p, R^{prep} \rangle.$$

Here ρ is the selected route, a is authentication, ℓ_h is a bit mask for raw-key, error-correction, and privacy-amplification completion, A is the relay acknowledgement mask, b records acceptance of an invalid-session acknowledgement, c is endpoint confirmation, u is stored status, x is expiry, e is link failure, t is the local event count, z is active/delivered/aborted status, R is a delivery receipt, and v records whether an abort had a valid cause when taken. The new field p records a pending prepared delivery, and R^{prep} stores its earlier precondition snapshot.

For a correct hop, the successive masks are 0, 1, 3, 7. Mask 5 represents raw-key and privacy-amplification completion without error correction; it is introduced by an ordering mutant. Relay identity validation is coarse: the model records whether a foreign acknowledgement was accepted, rather than modeling a message buffer with explicit session identifiers.

Each session uses a key index $k(i)$. Independent-key configurations assign one index per session. The shared-key configuration assigns both sessions the same index. This is a single-candidate allocation abstraction, not a complete inventory of hop keys and application keys.

3.2 Transition rules

The transition relation $T \subseteq S \times A_c \times S$ is generated by the Python function `successors`. Only active sessions produce operational actions. Route selection initializes hop masks. Authentication sets a. Link actions add the next permitted completion flag to one hop. Acknowledgement actions add one relay bit after all hop masks equal 7. Confirmation, storage, and delivery follow their corresponding guards. Figure 1 summarizes this structure.

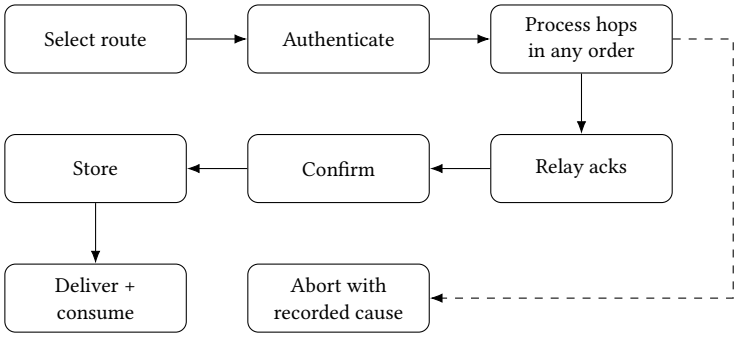


Fig. 1. Normal service lifecycle. The dashed branch illustrates a failure path; failure, expiry, and budget conditions are checked at their enabled states.

Every ordinary action advances t by one up to a finite deadline B. A delay action, where enabled by configuration, advances only this budget. When $t = B$, normal session actions stop and abort is enabled. A link-failure action sets e, after which abort is the only action for that session. Expiry may occur after storage; it enables abort and disables correct delivery. Another session consuming the shared candidate also permits abort for the waiting session.

An expired session can still perform some non-delivery actions before abort. This does not affect the claimed delivery safety, since the delivery guard rejects expiry. It does mean the model does not impose an immediate-abort requirement. Such a requirement would need a further transition or state contract.

In atomic mode, the correct delivery action takes a snapshot of the preconditions and changes status to delivered. The same atomic action sets $C[k(i)]$ to true. The snapshot remains available after the update. There is no key replenishment or reset of the consumed vector. A second active session requesting the same consumed candidate cannot deliver it. The omission mutant removes this consumed update, allowing the lifecycle obligation to fail. Split and reserved modes are defined separately below.

3.3 Termination semantics

The graph contains every reachable state under the configured finite domains. A terminal state has only delivered or aborted sessions. If a reachable state has no operational outgoing edge, the explorer adds a self-loop. A terminal loop represents completed behavior. A nonterminal loop represents a deadlock and causes progress failure.

Normal execution does not include arbitrary stuttering at every active state. The budgeted delay action represents finite waiting. The retry mutant explicitly adds a nonterminal self-loop without advancing the budget. No fairness restriction removes that loop. Therefore the eventual-termination result applies to this operational graph, not to an unrestricted stuttering specification or a general asynchronous scheduler.

3.4 Split delivery and reservation repair

The atomic action hides a common implementation boundary: a service can check a key in one operation and commit delivery in another. This resembles the time-of-check/time-of-use weakness category CWE-367 [8]. We therefore make the boundary explicit in two additional reference modes. These modes are executable alternatives of the same model, not source-code transformations of an external controller.

In split mode, `PrepareDelivery` checks the original delivery guard and records R^{prep}. It sets p but does not consume or reserve the key. A later `CommitDelivery` returns it and sets $C[k]$. Crucially, another session can prepare the same candidate between these actions. An expiry event can also occur while a prepared session waits. The unsafe split commit trusts the earlier decision and permits those stale decisions to complete.

The monitor records freshness at the actual commit, not merely at preparation. If the key has expired or has already been consumed, $R[fresh]$ is false. Thus a false earlier check cannot be hidden by copying its value into the final receipt. Duplicate delivery is additionally detected by P9, which counts delivered sessions sharing a candidate. The two observations identify related but distinct failures: a stale precondition and repeated resource release.

Reserved mode adds a model-level repair. Preparation requires $O[k] \in \{-1, i\}$ and assigns $O[k] = i$ atomically. Commit requires ownership, no consumption, and no expiry. A successful commit consumes the candidate and releases ownership. A valid abort also releases a reservation held by that session. Other requests can continue their link processing, but cannot prepare delivery against the reserved candidate. The repair does not replenish key material or guarantee that every competing request succeeds.

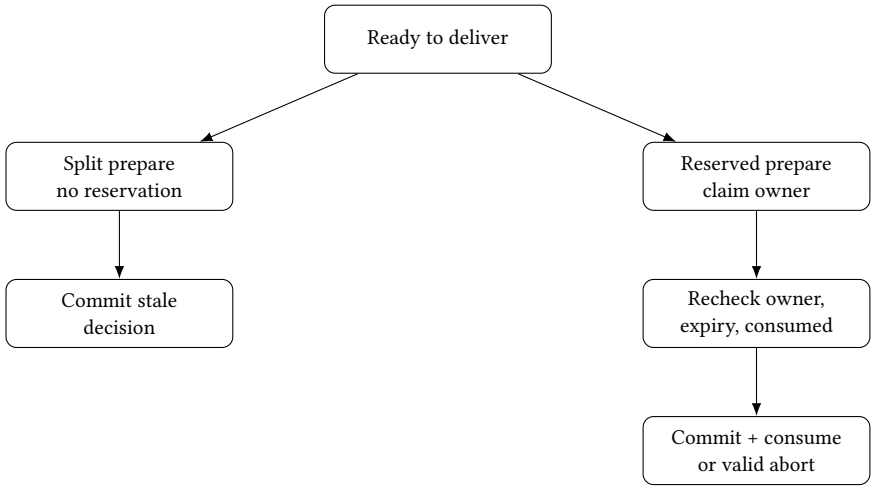


Fig. 2. Executable delivery alternatives. The split branch deliberately exposes the gap between check and commit. The reserved branch repairs that reference-model gap.

For reserved mode, the single-use argument extends naturally. At most one session holds $O[k]$. A commit requires that ownership and an unused candidate, and atomically marks the candidate consumed. Aborts release ownership but do not clear consumption. Therefore a completed delivery cannot be followed by another successful delivery of that candidate in the correct reference transition system. This manual argument relies on the atomic reservation operation; it is not a database isolation proof.

The experiment also includes separate-key controls. If each session has its own candidate, duplicate delivery across sessions is no longer possible through key sharing. Split mode can still fail freshness after expiry. This control separates the shared-resource race from the temporal gap within a single request. Figure 2 shows the corresponding decisions.

4 Contracts and Checking Algorithm

4.1 Delivery and lifecycle contracts

Write d_i for delivered status and ab_i for aborted status. Let $R_i[j]$ denote the recorded delivery precondition. The safety contracts are given in Table 3. All are checked over reachable states. Post-processing order is checked even in states that never deliver; this prevents an invalid intermediate ordering from disappearing after a later correction.

Table 3. Contract suite. Delivery predicates refer to the pre-update receipt.

Contract	Requirement
P1	AG $\bigwedge_i (d_i \Rightarrow R_i[auth])$.
P2	AG $\bigwedge_{i,h} (PA_{ih} \Rightarrow EC_{ih})$, including recorded delivery ordering.
P3	AG $\bigwedge_i (d_i \Rightarrow R_i[agreement])$. All required relay bits are present and no invalid-session acknowledgement was accepted.
P4	AG $\bigwedge_i (d_i \Rightarrow R_i[confirmation])$.
P5	AG $\bigwedge_i (d_i \Rightarrow R_i[fresh])$. Fresh means not expired and not consumed before delivery.
P6	AG $\bigwedge_i (d_i \Rightarrow R_i[full])$. The full condition requires authentication, all hop masks equal to 7, agreement, confirmation, storage, and freshness.
P7	AG $\bigwedge_i (ab_i \Rightarrow v_i)$. The cause was recorded at the abort event.
P8	AF $terminal$ from the initial state, without fairness filtering.
P9	AG$\big[\bigwedge_i (d_i \Rightarrow C[k(i)]) \ \wedge \ \bigwedge_k |\{i : d_i \wedge k(i) = k\}| \le 1\big]$.

P6 overlaps several delivery predicates intentionally. The smaller predicates identify a failing responsibility. The full predicate provides an aggregate service boundary. P2 adds intermediate-state coverage that a delivery-only check lacks. P7 records the reason at the event rather than inferring it from a later counter value. P9 separates a correct precondition from the required persistent update after delivery.

A valid abort cause is a recorded link failure, expiry, exhausted local budget, or an already consumed candidate. It is not a claim that the implementation has diagnosed the physical reason for a failure. Likewise, endpoint confirmation is one Boolean report; the model does not compare real key bits. These distinctions define the meaning of the contracts.

4.2 Exhaustive exploration

The explorer uses breadth-first discovery and stores one predecessor action for each newly reached state. It evaluates safety predicates across all reachable states. For a violated predicate, the first discovered violating state yields a shortest prefix in the model's unit-action distance. This does not imply a shortest real execution time or a unique diagnosis. The report includes the action sequence and the final abstract state.

Progress uses the standard least fixed point

$$W_0 = \{s \in S : terminal(s)\}, \qquad W_{j+1} = W_j \cup \{s : Succ(s) \subseteq W_j\}.$$

The self-loop convention makes $Succ(s)$ nonempty. At convergence, the initial state belongs to W exactly when all infinite paths in the totalized finite graph eventually reach a terminal state. A reverse-edge worklist decrements the number of successors not yet known to terminate. Each state and edge is processed a bounded number of times.

If the initial state is outside W, the checker repeatedly selects a successor outside W. Finiteness guarantees a repeated state, giving a prefix and a cycle. This lasso is a valid nontermination witness. Unlike the safety prefix, it is not claimed to be shortest. The distinction is recorded in the JSON output.

The exploration and progress passes have $O(|S|+|T|)$ graph cost, apart from state construction, hashing, and property evaluation. State and edge storage is also $O(|S| + |T|)$. Evaluating session and hop predicates adds a factor proportional to the represented session and route sizes. A configured state cap raises an exception instead of reporting a pass; an incomplete exploration has no verification verdict.

4.3 What a passing result establishes

Two short arguments explain important assumptions. For correct atomic and reserved configurations with positive route lengths and finite nonnegative budgets, define

$$V(s) = \sum_{i:z_i=active} (B_i - t_i + 1).$$

Every nonterminal correct operational action either increments one active session's counter or terminates that session. Thus it strictly decreases V. An exhausted active session has an abort action; a non-exhausted session has a normal next action or a valid abort. Consequently there is no nonterminal deadlock and no infinite sequence of baseline operational actions. Preparing and committing a delivery each spend one budget unit. Terminal self-loops preserve completion. This is a manual ranking argument for the operational model, not a fairness argument for an implementation. It also explains why P8 passes by construction when the budget rules are preserved. The retry and missing-timeout variants deliberately break its premises.

For shared-key safety, $C[k]$ is initially false, correct delivery requires it false, and that same atomic action sets it true. No action resets it. Hence a second delivery of the same candidate is disabled. This argument depends directly on atomicity. The omitted-consumption variant invalidates the update premise. Receipt freshness remains a pre-update fact, so consuming a correctly delivered key does not falsify P5. These arguments describe the reference transition relation; they are not mechanically checked theorem proofs.

For the generated finite graph, the algorithm checks the stated state predicates and the fixed-point progress condition. This is exhaustive reachability within the chosen abstraction, rather than a search over one sampled execution. It is not a proof for every relay count. It is also not a proof that the Python explorer is free of defects. The artifact provides targeted tests, full witness replay, and an independent graph-level cross-check to reduce, rather than eliminate, that risk.

5 Public Measurements and Measurement-Driven Verification

5.1 Evidence source and scope

The empirical arm uses the pinned IrelandQCI v2 record, DOI `10.5281/zenodo.21792844` [6]. The source describes measurements from the TCD/CONNECT testbed. It covers Toshiba MU coexistence experiments, standalone Toshiba LE production and research variants, and ID Quantique Clavis3 and ClavisXGR characterisation. The research variant also supplies perturbation conditions. These are measurements collected by the source researchers; this study reuses them and does not claim to have operated their equipment.

All 39 deposited files are acquired with their original bytes preserved. Their MD5 values are checked against the Zenodo record, and SHA-256 values are recorded locally. The package includes the original README, data dictionary, methodology, and CC BY 4.0 license. Source-file name and row number are retained for every normalized observation. Each observation ID is derived from its source-file digest and line number, allowing every derived capacity to be traced back to an original record.

The measurement fields include QBER as a fraction, secure key rate in bits per second, and added optical attenuation in dB. Depending on the campaign, additional fields identify platform, fibre setting, injected signal power, run, or perturbation condition. No source CSV contains a timestamp column. A row sequence is therefore not reinterpreted as a continuous network trace. In particular, sweep ordering and perturbation labels are not sufficient to reconstruct packet timing, key-manager acknowledgements, or controller delays.

The source documentation distinguishes communication-timeout rows removed during characterisation cleaning from zero secure-key-rate rows that remain in the published data. Accordingly, this study does not infer an outage process from the count of zeros. QBER is analyzed as a measured covariate. It is not converted into a universal security threshold: the platforms use different implementations and protocol families, and the published secure key rate already represents the reported output metric.

5.2 Data audit and overlap handling

The 23 CSV files contain 54,531 rows. All numeric QBER, SKR, and attenuation fields parse without rejection under the declared domain checks: finite values, nonnegative SKR, and QBER between zero and one. These checks establish structural suitability for this analysis, not calibration accuracy or cryptographic validity of the source devices.

A multiset comparison identifies a material overlap. Every row of the auxiliary production CSV marked `25samples` occurs in the full production sweep with at least the same multiplicity. The overlap is exactly 6,024 rows. Counting both files as independent evidence would inflate the production-system contribution. We preserve both originals but exclude the auxiliary subset from the primary analysis. This leaves 48,507 records across 22 CSV files and 1,331 operating-point groups.

Repeated values within an original file are not automatically removed. Quantized device outputs can legitimately repeat, and the source does not provide identifiers proving that each repeated value is a duplicate acquisition. The exclusion is therefore limited to the demonstrable complete auxiliary-file subset. This rule avoids both double counting the published subset and silently discarding legitimate repeated observations.

The audit also compares source summary records with the CSVs. Row counts agree for all 23 CSV files. Characterisation maxima agree. Two published minimum-rate summaries match the positive-rate minimum rather than the all-row minimum: Clavis3 reports 10 bit/s while its CSV includes zero, and the research sweep reports 29 bit/s while its CSV includes zero. Our calculations use the actual CSV values, including zeros, and record this convention explicitly. We do not overwrite the source summary files.

There are 533 zero-SKR rows in the primary analysis. Twenty-six published QBER observations exceed 0.1; all occur in the 12 dBm injected-signal, 50 km added-fibre file and have zero SKR. The maximum published QBER is 0.841. These values are retained and flagged. No decimal-place correction is guessed. They contribute no generated key units through the SKR mapping because their reported SKR is zero.

Table 4 reports the primary coverage and measured rate ranges. Platform row counts are unequal. Attenuation coverage, campaign conditions, and sampling densities also differ. The table is a source inventory, not a ranking of device performance under one controlled common workload.

5.3 Measured envelopes and perturbation summaries

Figure 3 shows descriptive median envelopes. Standalone platform curves are grouped by attenuation. Coexistence curves are separated by added-fibre setting and injected signal power. No missing attenuation point is numerically

Table 4. Primary published observations after excluding the exact 6,024-row auxiliary subset. Rate ranges are descriptive across different operating conditions, not device rankings.

Platform	Rows	Zero SKR	SKR range (bit/s)
Clavis3	769	7	0–2,441
ClavisXGR	23,409	0	143–6,755
Toshiba LE prod.	17,274	0	541–2,758,846
Toshiba LE research	3,162	83	0–2,169,195
Toshiba MU	3,893	443	0–354,598

interpolated for the analysis. Zero median rates are absent from the logarithmic rate panels, but all zero records remain in the normalized data, capacity mapping, and reported counts.

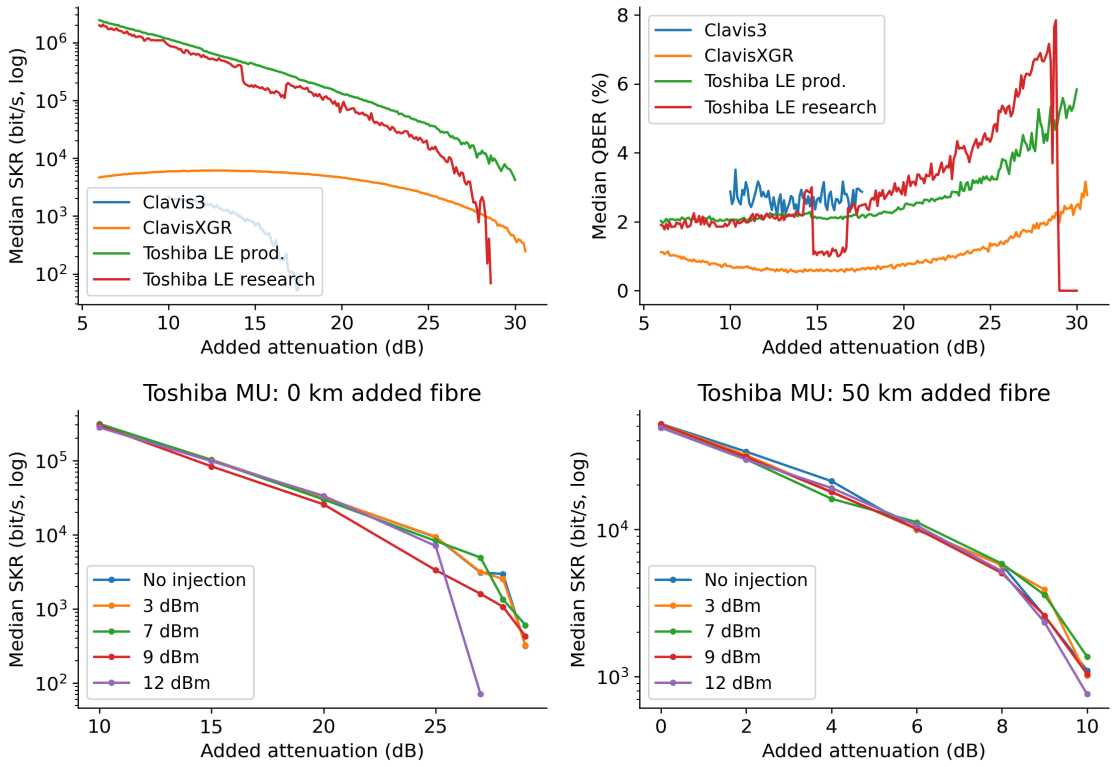


Fig. 3. Measured operating envelopes from the published primary CSVs. Top: standalone rate and QBER medians by added attenuation. Bottom: Toshiba MU coexistence rate medians separated by fibre setting and signal injection. These are descriptive source-condition summaries, not matched device rankings.

The perturbation data are summarized at the three shared attenuation settings, 9, 12, and 15 dB. For each condition, its median SKR is divided by the baseline median at that same attenuation. This controls the reported comparison for attenuation while retaining the source's condition labels. It does not reconstruct the timing of individual perturbations or establish an independent treatment-effect estimate.

Table 5. Median measured SKR divided by the baseline median at the same attenuation. Labels are source experimental settings, not reconstructed event times.

Condition	9 dB	12 dB	15 dB
baseline	1.000	1.000	1.000
1s_settling	0.847	0.834	0.804
2s_settling	0.838	0.848	0.825
5s_settling	0.826	0.859	0.815
20s_settling	0.863	0.876	0.810
continuous	0.770	0.757	0.720
continuous_noisy	0.750	0.733	0.695
random	0.772	0.739	0.735

Table 5 shows ratios below one for each non-baseline condition. For example, the continuous-noisy condition has ratios 0.750, 0.733, and 0.695 at 9, 12, and 15 dB. These measured operating differences can change a supply scenario. They are not themselves observations of a controller fault, and the analysis does not label every low-rate sample as a service failure.

5.4 From measured rate to a declared service scenario

A transparent bridge converts a measured secure key rate r_i into a hypothetical finite accumulation scenario. Let W be an assumed accumulation window in seconds and B the assumed number of bits per application key unit. The number of complete units available at the start of a batch is

$$C_i(W, B) = \left\lfloor \frac{r_i W}{B} \right\rfloor .$$

The primary bridge uses $B = 256$ bits and $W \in \{0.001, 0.01, 0.1, 1\}$ seconds. Sensitivity calculations also use 128 and 512 bits. These are service segmentation assumptions, not measurements of actual application key size or a prescription for a cryptographic scheme. Decimal arithmetic is used to avoid a floating-point boundary changing a floor result.

The bridge assumes the reported rate is sustained over the hypothetical accumulation window, starts with no pre-existing inventory, and performs no refill during the subsequent batch. It therefore produces measurement-driven scenarios rather than replay of actual inventory history. The source has no timestamps from which such an inventory history could be recovered. The rate window is measured in seconds; the controller expiry/deadline budgets below count model events. These quantities are never equated.

For $N \in \{2, 4, 8\}$ identical requests, define $\widehat{C}_i = \min(C_i, N)$. Capacity above N is saturated because at most N unit deliveries can occur in a no-refill batch. Under identical unit requests and count-based predicates, replacing any capacity $C \geq N$ by N preserves the relevant availability decisions before all requests finish. This is a manual abstraction argument for the stated resource model. It is not a proof for variable key sizes, per-request identities, replenishment, or arbitrary queue disciplines.

The primary bridge produces 582,084 records: one for each primary observation, four windows, and three request counts. Each stores observation ID, window, unit size, original token count, and saturated capacity class. The broader sensitivity calculation performs 1,746,252 transformations across three unit sizes. It stores grouped counts rather than falsely treating those repeated transformations as new independent measurements.

Figure 4 separates the source measurement from assumptions added by the experiment. All seventeen possible saturated $(N, \widehat{C})$ pairs for the selected workloads are observed somewhere in the primary bridge. This establishes that

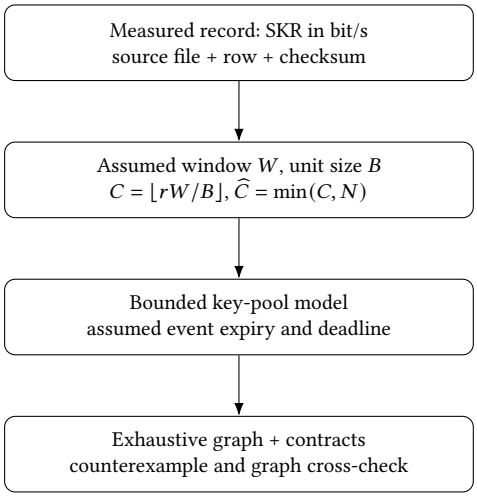


Fig. 4. Traceable measurement-to-model bridge. Measurement provenance is retained, while workload and controller-budget assumptions remain explicit.

every resource-capacity class studied below has measurement associations. It does not establish that every controller budget or workload was observed on the source testbed.

Table 6. Measurement-row associations with capacity classes for a 10 ms assumed accumulation window, 256-bit units, and four requests. Counts describe published sampling, not outage probabilities.

Platform	C=0	C=1	C=2	C=3	C≥4
Clavis3	769	0	0	0	0
ClavisXGR	23409	0	0	0	0
Toshiba LE prod.	978	566	372	301	15057
Toshiba LE research	217	55	43	20	2827
Toshiba MU	2459	590	119	316	409

Table 6 gives one declared scenario: four requests, 256-bit units, and a 10 ms accumulation window. Both Clavis groups have zero complete units in this window even though most or all of their source rows have positive SKR. This is insufficient accumulation for the chosen unit, not an observed equipment outage. The distinction prevents a conversion artifact from being presented as a measured failure.

5.5 Resource model and contract semantics

The measurement-driven model complements the earlier coordination model. It isolates key-pool allocation after earlier protocol processing has reported completion. It does not prove a formal refinement relation to all P1–P9 obligations. Its four contracts use separate names, I1–I4, so that resource evidence is not confused with full-controller verification.

A state is

$$s_p = \langle w, p, d, a, f, t, b \rangle,$$

where w is waiting requests, p is prepared requests, d is delivered requests, a is aborted requests, f is free units, t is a global controller-event count, and b is a sticky flag recording a stale commit. Initially $w = N$, $p = d = a = 0$, $f = \widehat{C}$, and

$t = b = 0$. Requests are identical and represented by counts. This symmetry quotient removes identities only because the transitions and predicates in this experiment do not distinguish them.

Every active operational action advances t by one, capped at deadline D. A tick action can advance time without allocating a unit. All units in a batch share the same modeled expiry budget L; a unit is fresh at a transition when $t < L$. The experiment uses $L \in \{3, 6, 12\}$ and $D \in \{6, 12\}$. These are explicit event-budget sensitivity settings. They are not derived from physical timing fields, which the source does not provide.

In atomic mode, a waiting request can deliver only while a free fresh unit exists and the deadline permits the action. Delivery decreases w and f and increases d. Split mode first prepares without reserving f, then permits a later commit that can trust an obsolete decision. That commit decreases f even if another prepared request already consumed the supply, or if expiry has intervened. It deliberately exposes the resource-level check/use error.

Reserved mode decreases f when preparing. Commit requires a prepared request, freshness, and an open deadline. Commit moves one request from p to d. An aborted prepared request returns its reservation only while that unit is still fresh; an expired unit is not returned as reusable supply. Waiting requests may abort when no free unit is available, the key has expired, or the deadline has closed. Thus a correct resource model need not serve every request when the declared supply is inadequate.

Table 7. Contracts for the measurement-driven resource abstraction.

Contract	Meaning
I1	$\mathrm{AG}(f \geq 0 \wedge d \leq \widehat{C})$. No negative supply or over-delivery.
I2	$\mathrm{AG}\,\neg b$. Every actual commit uses fresh material at that event.
I3	$\mathrm{AG}(w + p + d + a = N)$ with nonnegative request counts; reserved mode additionally requires $f + p + d \leq \widehat{C}$.
I4	$\mathrm{AF}(w + p = 0)$. Every unrestricted path eventually terminates the batch.

Table 7 states the checked claims. Each graph is explored exhaustively. Safety failures produce shortest unit-action BFS prefixes. Progress uses the earlier least-fixed-point construction. The finite event budget and termination actions ensure eventual completion of the batch under these rules, even if that completion includes aborts. A separate C++ cycle-elimination checker reads every exported resource graph and uses I4's progress bit. It still consumes Python-generated transitions and labels, so the same independence limitation applies.

5.6 Executed resource checks and results

Seventeen observed capacity classes, three modes, three expiry budgets, and two deadlines give 306 executed cases. All complete. Together they contain 52,611 configuration-counted states and 116,400 totalized edges. The C++ checker agrees on all 306 graphs. These are additional resource-model graphs; they are not extra hardware measurements and are not folded into the original 522-case grid count.

Atomic and reserved modes each pass all 102 resource cases. Split mode fails at least one contract in 61 of 102 cases. The other split cases remain in the denominator: zero supply can prevent any preparation, an adequate capacity can prevent over-delivery, and expiry or deadline settings can mask a stale commit. Table 8 separates the contracts rather than implying that every split setting has the same failure.

Figure 5 shows why measured capacity and assumed control behavior must be considered separately. A low measured supply maps to a constrained resource class. It does not force an unsafe outcome: the atomic and reserved reference

Table 8. Resource-model verification across observed capacity classes and assumed event budgets. Every case completes and agrees with the C++ graph checker.

Mode	Cases	Any fail	I1	I2	I3	I4
atomic	102	0	0	0	0	0
split	102	61	38	42	0	0
reserved	102	0	0	0	0	0

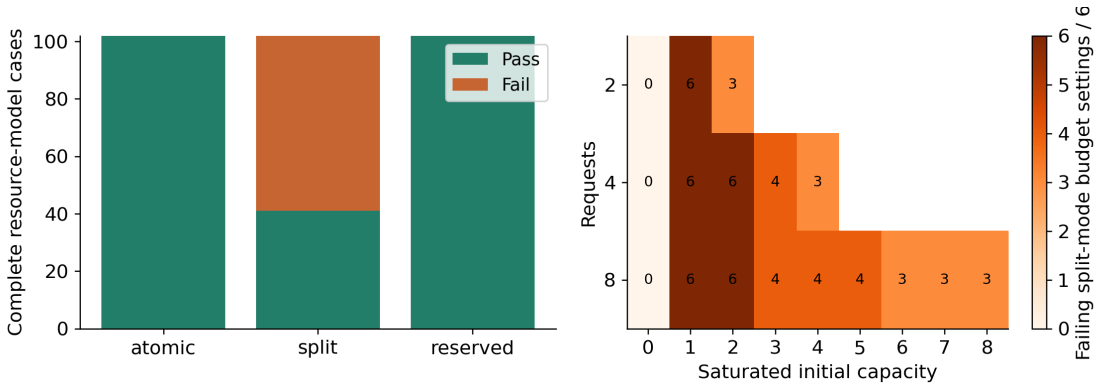


Fig. 5. Executed resource-model outcomes. Left: pass/fail cases by mode. Right: failing split-mode budget combinations for each observed workload/capacity class. Empty cells are outside that workload's saturated domain.

modes can abort safely. The deliberately unsafe split mode can instead over-commit or use stale material. No such modeled failure is reported as an observed defect of a Toshiba or ID Quantique product.

The row-to-profile association file joins every source-derived capacity class to its executed mode and budget cases. It is suitable for tracing which measurements support a given operating regime. Its unsafe-profile counts are not field failure probabilities. The source sampling covers attenuation sweeps with unequal densities, and the controller workloads and budgets are chosen by this study.

5.7 Sensitivity to window, key unit, and sampling density

Figure 6 combines the measured perturbation summaries with the declared supply-window sensitivity. Increasing W can produce enough complete units from the same positive rate. Changing B changes that boundary. Both effects are consequences of the explicit conversion, not additional observations of the device over a newly measured interval.

Sampling density matters independently of the window assumption. The primary analysis groups observations by source file, condition, run, fibre setting, noise, and attenuation. An equal-operating-point summary averages the within-group adequacy fraction, assigning each group equal weight. The row-weighted version instead follows the published record density. Table 9 reports both, without selecting whichever appears more favorable.

For the production system at the 10 ms window, the four-unit adequacy fraction is 0.872 under row weighting and 0.633 under equal-point weighting. For the research system it is 0.894 and 0.513. These differences are not contradictory measurements. They reflect unequal counts across operating conditions. They show why a pooled dataset fraction should not be described as the probability that a deployed service has enough keys.

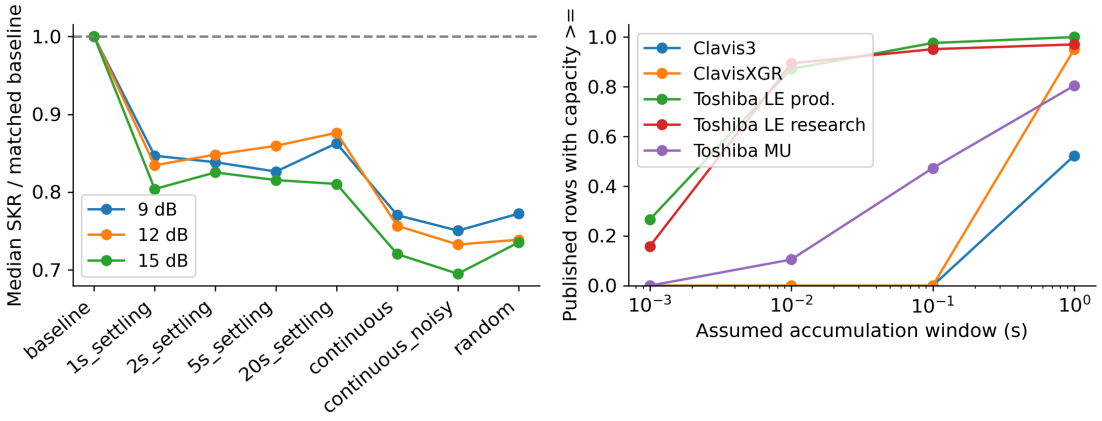


Fig. 6. Measured and assumed components. Left: measured median perturbation/baseline ratios at matched attenuation. Right: fraction of published rows producing at least four 256-bit units under each assumed window. The right panel describes sample associations, not time availability.

Table 9. Fraction of observations meeting four 256-bit units: row weighting versus equal weighting of operating-point groups. These are descriptive sampling summaries.

Platform	Window s	Row weight	Point weight	Groups
Clavis3	0.01	0.000	0.000	77
Clavis3	1	0.523	0.522	77
ClavisXGR	0.01	0.000	0.000	247
ClavisXGR	1	0.951	0.901	247
Toshiba LE prod.	0.01	0.872	0.633	241
Toshiba LE prod.	1	1.000	1.000	241
Toshiba LE research	0.01	0.894	0.513	686
Toshiba LE research	1	0.970	0.863	686
Toshiba MU	0.01	0.105	0.082	80
Toshiba MU	1	0.805	0.755	80

5.8 Measurement audit and interpretation of RQ1

The measurement audit verifies all 39 raw-file checksums, all 23 CSV row counts against published summaries, every one of the 582,084 primary capacity mappings, all resource-graph digests, and each stored resource counterexample. Seven added tests cover source checksums, subset exclusion, exact floor boundaries, selected capacity-saturation controls, freshness at actual commit, reservation release after abort, and graph-check agreement. Together with the original tests, twenty tests pass.

RQ1 is therefore answered at two levels. The real data establish operating-rate and QBER observations for the published experimental conditions. A reproducible, declared transformation maps those observations to finite supply classes, whose resource contracts are checked exhaustively under assumed controller policies and event budgets. What remains absent is an operational controller trace linking those device measurements to actual session scheduling, acknowledgements, storage transactions, expiry timing, or observed coordination faults. The paper retains that boundary rather than converting model evidence into deployment evidence.

6 Synthetic Benchmark Dataset

6.1 Dataset purpose and generation

VeriQKD-Bench is generated by a deterministic parameter design. Each case is a configuration of the same reference semantics. The full ordered design is stored before outcomes are analyzed. The configuration includes relay count, session count, route-length choices, failure and delay switches, local event budget, key-sharing policy, delivery mode, fault operators, targeted locations, and a state cap. No device log or network traffic record is imported.

A canonical configuration is serialized using sorted compact JSON. The first sixteen hexadecimal digits of its SHA-256 digest form the case ID. Duplicate configurations are merged, with family memberships retained. This prevents simple renaming from creating extra cases. The ID includes the state cap, because a resource limit changes the experiment even when it does not change the mathematical transition system. Distinct IDs can still produce identical reachable graphs. We therefore report exact graph-serialization diversity separately.

Dataset construction follows the practical goal of making configurations, properties, and expected outcomes inspectable, as in established verification benchmark practice [9]. The current dataset is not part of the PRISM suite and does not use PRISM to obtain its outcomes. It also does not claim the completeness or external maturity of that benchmark. Machine-readable metadata and integrity records support reuse in the spirit of the FAIR principles [10]; no external repository certification or archival identifier is claimed.

6.2 Parameter families

The structural family has 72 configurations. It crosses one to six relays with failure enabled/disabled, delay enabled/disabled, and three event-budget regimes. A route with $H = r + 1$ hops requires $B_{min} = 4H + 4$ ordinary actions for the shortest fault-free atomic success: route selection, authentication, three actions per hop, $H - 1$ acknowledgements, confirmation, storage, and delivery. The structural budgets are $B_{min} - 3$, B_{min}, and $B_{min} + 4$. The first is deliberately too short for success; it tests safe termination rather than service completion.

The concurrency family has 72 configurations. It includes two sessions with zero or one relay, and three sessions with zero relays. It crosses shared/separate candidate keys, failure enabled/disabled, atomic/split/reserved delivery, and short/slack budgets. Split and reserved delivery spend an additional action on preparation. Their budget regime uses the corresponding shortest successful path length. A per-session budget is not a global timer; interleaving another session does not advance a waiting session's local counter.

The first-order family has 288 configurations: twelve variants, three relay counts, two failure settings, two delay settings, and two budget regimes. One variant changes only an abort diagnostic label. The other variants alter operational behavior. These are contextual instances of eleven selected behavior changes, not 264 distinct defect types. The localized family adds 22 cases targeting selected sessions, hops, and relay acknowledgement positions. It checks that a location-specific injection affects the named participant rather than every component.

The combined family has 56 configurations. It evaluates all 28 pairs among eight selected operators at two relay counts. This is a selected higher-order study, not the full power set of every available operator. The operators include ordering, confirmation, freshness, consumption, invalid-abort, and retry changes. Their interactions may mask a delivery fault by terminating an execution earlier. Such a masking outcome is retained rather than removed from the results.

The route family has twelve configurations. It crosses three hop-count menus with failure and delay switches. Menus are $(1, 2)$, $(2, 3)$, and $(1, 3, 4)$. The model chooses a route length before processing begins. These are alternative

abstract service shapes. They do not constitute a physical branching topology, shared-link contention experiment, or route-optimization dataset.

Table 10. Synthetic benchmark coverage. A capped case has no verification verdict. State totals count each complete configuration separately, including repeated graph structures.

Family	Cases	Complete	Capped	Fail	State total
structural	72	63	9	0	747,691
concurrency	72	72	0	12	322,042
faults	288	288	0	166	562,181
localized	22	22	0	22	51,316
combined	56	56	0	56	12,876
routes	12	12	0	0	30,162

Table 10 reports observed completion and failure counts by family. The family totals are experimental counts. They are not statistically independent samples from all possible QKD services. The structural sweep and fault family deliberately share related parameter values, while the localized and combined families are selected extensions.

6.3 Records, labels, and unknown outcomes

Each input has a configuration JSON file. A completed outcome contains reachable state and edge counts, nine property values, successful-delivery reachability counts, core verification time, worker peak resident memory, safety counterexamples or progress lassos, and the independent graph-check result. A capped outcome records the cap, discovered and expanded state counts, elapsed core time, peak memory, and null property values. It has no complete graph checksum and no pass/fail verdict.

The complete graph format is intentionally small. The header gives the vertex and edge counts. Each vertex has a terminal flag and a safety-violation bit mask. Each subsequent edge is an integer source/target pair. State zero is initial. A gzip file preserves the graph for every complete case. Compression uses a fixed timestamp and no embedded source filename, allowing exact compressed-byte digests to be checked. The graph omits action text; action sequences remain in the outcome witnesses and sampled traces.

A flat CSV exposes the main fields for analysis. Property values are the strings pass, fail, or unknown. Unknown is not encoded as zero, because that could be mistaken for a successful or negative observation. A JSON schema checks the structural distinction between completed and capped records. Additional audit code checks semantic consistency, case IDs, checksums, witness actions, and reference outcomes.

6.4 Provenance and reuse limits

Table 11 defines the main records. The package also includes source-file hashes, environment metadata, generator scripts, a dataset card, and a file manifest. Regeneration can produce different timing and memory observations on another system. Structural graph counts and verdicts should match when the model source, configuration, and interpreter semantics are unchanged. A mismatch is an audit failure to investigate, not a reason to overwrite the reference silently.

The dataset contains no real key bits or personal information. It is not a dataset of real attacks. A researcher could use it to test graph algorithms or to develop property diagnostics, but should avoid training/testing leakage from closely related configurations or identical graph serializations. This paper trains no machine-learning model and reports no

Table 11. Core dataset fields and their interpretation.

Field or file	Interpretation
`case_id`	Digest of the canonical experimental configuration.
`status`	Complete exploration or capped exploration.
`properties`	Nine Boolean verdicts for complete cases; null for every capped verdict.
`states`, `transitions`	Reachable graph size, with totalization loops included in edges.
`witnesses`	Shortest BFS safety prefixes or valid, not necessarily shortest, progress lassos.
`states_with_delivery`	Number of reachable states with at least one completed delivery.
`seconds`	Core exploration/checking duration, excluding graph export and C++ checking.
`peak_rss_kib`	Peak memory of the Python worker; Linux KiB units.
`graph_sha256`	Exact compressed graph-file digest.
`random_traces.jsonl.gz`	Seeded executions with actions, observed failures, outcome, and loop index.

held-out predictive accuracy. Dataset reuse should preserve the explicit synthetic origin and the difference between an operator type and its contextual instances.

7 Experimental Protocol

7.1 Tool assistance and evidence generation

ChatGPT/Codex assisted with study scripting, independent-specification drafting, controller-code development, experiment execution, data-analysis code, literature discovery, and manuscript revision. Original hardware records were downloaded from the credited source rather than generated. Every reported numerical result was obtained from the supplied executed programs. Checks include raw-file checksums, unit tests, exhaustive graph checks, separate TLC runs, replay of stored witnesses, and SQL event-history inspection. AI-generated prose or expectations were not used as a result oracle. The accompanying provenance note records the requested tasks, tool role, and verification steps. Tool assistance does not constitute an independent replication or a correctness proof.

7.2 Frozen design and execution policy

The Python implementation performs breadth-first reachable-state exploration followed by safety evaluation and a reverse-edge universal-eventuality fixed point. Each grid case runs in a fresh worker process. Cases run sequentially. There is no parallel worker speedup in the reported measurements. The controller stops a case at its declared state cap and records an unknown outcome rather than extrapolating a verdict.

Structural and route cases use a cap of 100,000 states. Concurrency and most fault extensions use 60,000 states; the precise per-case configuration is authoritative. Location-specific hop and relay cases use the default 100,000-state cap. These limits are finite experimental controls, not physical network capacity limits. No complete-case rate is reported without also reporting the capped count.

The initial grid is deterministic. Randomness appears only in the sampled execution-testing experiment. Runtime metadata record Python, Linux, CPU information, and the C++ compiler version. The experiment environment used Python 3.12.14 on Linux x86-64 with a runtime-provided Intel Xeon Platinum 8573C CPU model. Nine logical CPUs were visible, but the experiment workers were sequential.

7.3 Graph-level cross-check

For every completed grid case, a separate C++ program reads the exported graph. It traverses from state zero, verifies the absorbing terminal convention, aggregates the supplied safety labels, and searches the reachable nonterminal subgraph for a cycle. The cycle search uses indegree elimination. If nonterminal vertices remain after eliminating zero-indegree vertices, a reachable cycle exists. This differs from the Python checker's reverse least-fixed-point implementation.

For a finite totalized graph with absorbing terminal states, a path that never terminates exists exactly when a reachable nonterminal cycle exists. Thus the two progress computations should agree. The cross-check also compares reachable vertex and edge counts. It is a useful control against graph-checking implementation errors. However, it consumes the Python-generated graph and safety labels. It cannot independently establish that the transition generator matches a QKD service or that a supplied label expresses the intended safety requirement. We therefore call it a graph-level cross-check, not an independent verification of an implementation.

7.4 Fault and property-subset comparisons

The first-order analysis retains every completed contextual instance of an operational variant. The diagnostic-label control is reported separately. A context with no observed property violation is a survivor for that context. It is not assumed to be semantically equivalent. A short budget may prevent the faulty behavior from reaching delivery, and a local fault may not matter when its related requirement is unreachable.

Property-subset comparisons use exactly the same full-exploration outcomes. Four subsets are evaluated: P6 alone; P1–P7; P1–P7 with P9; and the full suite. A context is detected by a subset if at least one property in that subset fails. No graph is re-generated differently for a subset. This isolates the information contributed by the obligations, rather than measuring a time-saving early-exit checker.

Mutation analysis is widely used to examine sensitivity to selected faults [11]. Here the faults are hand-authored with knowledge of the contracts. The analysis therefore does not estimate general bug-detection accuracy. It also does not rank mature model checkers. Both subset analysis and enabled-action testing use the same reference transition relation.

7.5 Seeded enabled-action testing

Thirty additional scenario configurations provide exhaustive reference verdicts for the testing comparison. They include each first-order variant at one and two relays, and six two-session delivery-mode controls with shared or separate candidate keys. Missing-timeout scenarios use a deliberately short budget so that the missing timeout action is reachable. Other scenarios use enough budget for their normal service path. Scenario configurations are stored independently from the main grid.

Each scenario uses two schedulers and twenty independent deterministic seed streams. Each seed trial generates 100 episodes, giving 120,000 episodes overall. The seed stream is derived from a fixed prefix, scenario ID, scheduler name, and seed index. Trace budgets of 10, 50, and 100 are prefixes of the same seed trial, allowing a monotonic within-trial comparison. They are not three independently sampled datasets.

The uniform scheduler selects uniformly among enabled model actions. The progress-biased scheduler assigns weights 0.02 to link failure, 0.05 to expiry, 0.10 to delay, and 1.0 to other actions, then normalizes. These weights encourage deeper executions. They are not measured failure rates or a calibrated QKD stochastic model. Changing the scheduling policy changes testing hit rates, while exhaustive reachability remains unchanged.

Each episode stops at a global terminal state, a nonterminal deadlock, a repeated full nonterminal state, or 200 actions. Safety predicates are monitored after every action. A deadlock supplies a nontermination witness after totalization. A repeated nonterminal state supplies a cycle that can be repeated under the unrestricted operational semantics. A finite execution without such a witness does not prove progress. The cutoff is therefore not treated as a liveness pass.

A seed-trial hit means at least one valid failing witness within the trace budget. We report both seed-trial hit counts and scenario coverage. Selected per-scenario rates use descriptive 95 percent Wilson intervals [12]. These intervals concern the configured pseudorandom experiment. They do not estimate the probability that a deployed network fails, nor are contextual model instances treated as a random sample of production defects. The study avoids population-level significance claims for this constructed benchmark.

7.6 Performance measurements

Performance measurements run after the grid and testing jobs have finished. Ten settings are used: relay chains with one through seven relays, plus two-session shared-key atomic, split, and reserved settings with one relay. Each setting runs in eight fresh processes. The first is discarded and seven are retained, giving 70 measured runs. The discarded run reduces some cache-transient effects; a fresh interpreter is still used for every repetition.

The core timer includes graph construction, safety checking, progress checking, and witness construction. It excludes the subsequent delivery-reachability summary passes, portable-graph export, gzip compression, and C++ checking. Peak RSS is obtained from the Python worker process. It excludes the separately executed C++ child. The repeated runs do not export graphs or invoke the cross-checker. Median, minimum, and maximum time are reported with median peak RSS. No statistical speedup claim against another tool is made.

7.7 Audit and test controls

The original thirteen automated tests exercise the baseline, delivery snapshots, abort authorization at the budget boundary, shared-key reuse, localized and combined faults, expiry at split commit, reservation release, cap behavior, safety witness replay, label-only equivalence, and progress checking with an independent cycle algorithm. The complete dataset audit then replays every stored exhaustive witness and every sampled trace action.

Replay checks more than whether a JSON file is readable. At each action, the audit reconstructs the enabled successors and requires the named action to identify the expected next state. A safety witness must finish in a state violating its named predicate. A progress lasso must close and contain no terminal state in its loop. A sampled trace's reported violations must equal the monitor's replayed violations and must be among the corresponding exhaustive scenario failures. Case IDs and compressed graph digests are also recomputed.

These controls address internal consistency and some algorithmic mistakes. They do not make the reference model or the predicates independently authoritative. A specification error shared by generation and replay can survive the audit. The interpretation of the results keeps that common-source limitation explicit.

8 Synthetic Benchmark Results

8.1 RQ2: Coverage, outcomes, and non-vacuity

The frozen grid contains 522 canonical configurations. Exhaustive exploration completes for 513 cases; 9 reach the declared state cap. Among complete cases, 257 satisfy all contracts and 256 violate at least one. These totals include

intended fault and unsafe-split controls. A failing negative control is not a failure of the experiment procedure. Table 10 separates the families and unknown outcomes.

The complete cases contain 1,726,268 configuration-counted vertices and 5,412,258 edges. The largest complete grid case has 94,658 states. There are 386 distinct exact exported graph serializations. The difference between 513 complete cases and 386 serializations demonstrates that configuration count is not behavioral diversity. Some variations have no reachable effect in a context, and the label-only control deliberately preserves state observations.

All nine capped cases are delayed structural configurations at five or six relays. Each reaches the 100,000-state limit. Their properties remain unknown. This is an important boundary: the same reference design can be tractable without explicit delay combinations and exceed a declared cap when those combinations are included. No passing claim is made for a capped instance, even if a smaller related configuration passes.

The 63 complete structural cases satisfy all contracts. Successful delivery is unreachable in all 21 completed short-budget structural cases. It is reachable in all 21 completed exact-budget cases and all 21 completed slack-budget cases. Thus the short-budget cases demonstrate safe termination only. They are not used as evidence that the service can complete its intended delivery within an inadequate budget.

Across complete cases with no fault operator and atomic or reserved delivery, 123 controls satisfy all contracts. This includes the route-length alternatives and correct concurrency modes. The separately unsafe split mode is not included in that control count. Figure 7 summarizes the pass/fail/unknown distribution and the property-subset results used below.

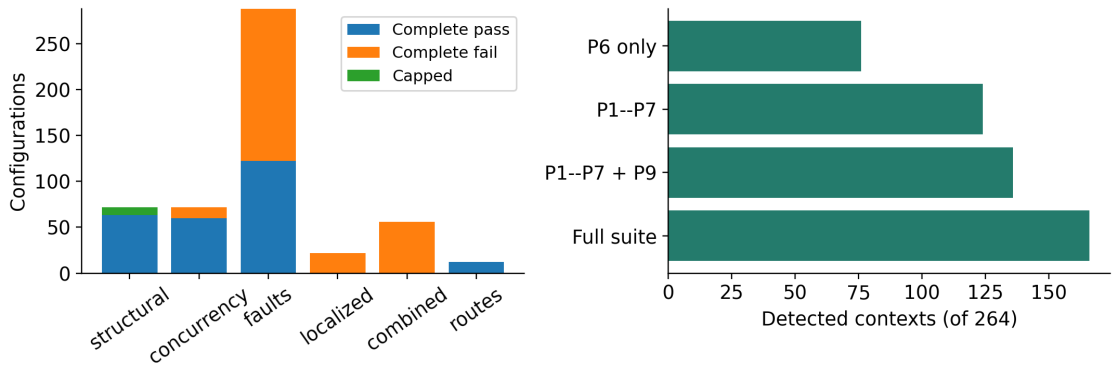


Fig. 7. Dataset outcomes and property-subset detection. Unknown cases remain separate. State-related totals elsewhere count complete configurations, not independent deployed systems.

8.2 RQ3: Fault contexts, budgets, and locations

The operational first-order analysis contains 264 completed contexts after excluding the diagnostic-label control. The full suite detects 166 of them. The other 98 contexts have no violation under their configured semantics. These survivors remain in the denominator. Reporting only contexts in which a faulty behavior was already known to reach its target would give a more favorable but less informative result.

Budget sensitivity explains much of this difference. There are 132 short-budget and 132 slack-budget operational contexts. Forty short-budget contexts fail, compared with 126 slack-budget contexts. The short budget prevents many

delivery-stage errors from reaching delivery. By contrast, invalid abort can fail immediately and post-processing order can fail before the full service completes. Missing timeout is especially sensitive to whether a budget can be exhausted without another valid termination cause.

Table 12. Context-dependent first-order results. A surviving context is not automatically an equivalent mutant.

Operator	Contexts	Detected	No violation
expired_delivery	24	12	12
invalid_abort	24	24	0
missing_timeout	24	18	6
omit_consume	24	12	12
retry_loop	24	12	12
skip_ack	24	16	8
skip_auth	24	12	12
skip_confirm	24	12	12
skip_ec	24	24	0
skip_store	24	12	12
wrong_session_ack	24	12	12

Table 12 shows all operator counts. Invalid abort and skipped error correction are detected in all 24 contexts for each operator. Missing timeout is detected in 18. Skipped acknowledgement is detected in 16, while several delivery-related operators are detected in 12. This variation does not establish that one class is intrinsically easier to detect in real software. It reflects where its affected transition lies in these bounded models.

The diagnostic-label control has 24 completed contexts and no property failure. Every one has a matching baseline graph serialization, giving 24 exact matches. This establishes equivalence under the exported state-observation boundary for these instances. A contract over exact action messages could distinguish the changed label. The conclusion is therefore narrower than protocol equivalence under every possible observer.

All 22 location-specific contexts fail at least one contract. The targeted-hop test confirms that the post-processing error occurs at the configured hop. Targeted-session cases expose the chosen session rather than applying the fault to all sessions. All 56 selected pair contexts also fail. These results show that the selected obligations remain sensitive under those combinations; they do not show coverage of every fault pair, every possible location, or real multi-defect interactions.

Figure 8 shows the property-level overlap. Delivery errors often violate both a specific obligation and P6. Invalid abort is isolated by P7. Omitted consumption is detected by P9, even when the pre-update delivery receipt is valid. Progress faults require P8. This division supports diagnosis without treating overlapping detections as independent successes.

8.3 RQ4: What separate contract groups add

On the same 264 complete operational fault contexts, P6 alone detects 76. P1–P7 detect 124. Adding P9 increases detection to 136. Adding P8 gives 166. Table 13 reports the exact finite-set counts. Every subset is evaluated on the same stored graphs and outcomes, so the difference is due to the observation set rather than a change in exploration strategy.

The added detections have a concrete interpretation. Intermediate post-processing order and abort authorization are not implied by a delivered-key assertion. Persistent consumption is a post-event obligation and can fail despite

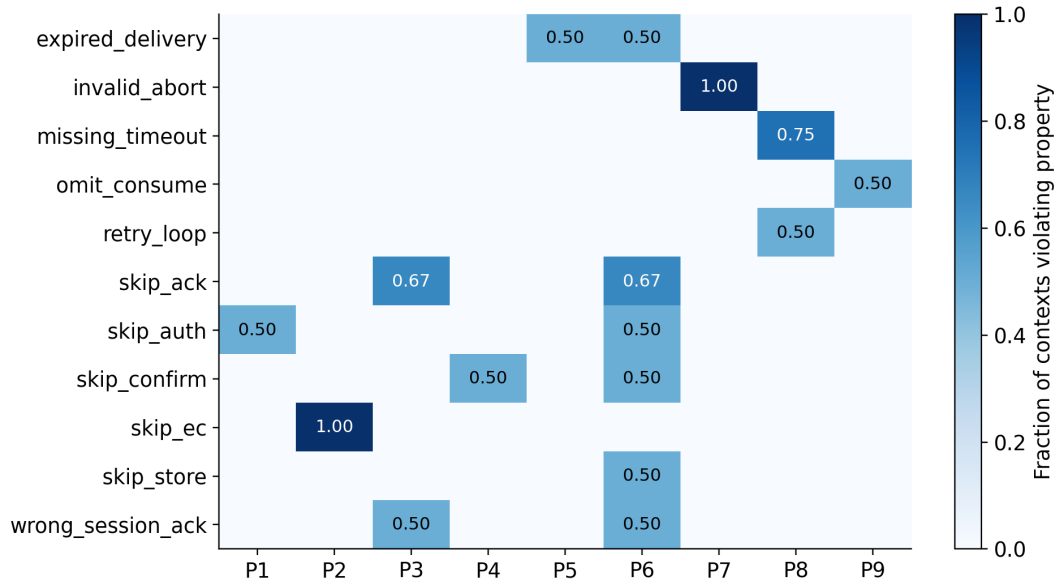


Fig. 8. Fraction of complete first-order contexts violating each property, by operator. Context masking remains visible; a blank cell is zero, not an omitted experiment.

Table 13. Detection over complete first-order fault contexts, excluding the diagnostic-label control. Contexts with no observed violation remain in the denominator.

Property subset	Detected	Evaluated
P6 only	76	264
P1–P7	124	264
P1–P7 + P9	136	264
Full suite	166	264

a correct delivery receipt. A nonterminating path need not deliver anything at all. The full suite therefore observes obligations that a delivery-only check cannot directly express at that event boundary.

The comparison is a sensitivity ablation, not an execution-time optimization. The implementation computes the full set of outcomes before subset analysis. It does not show whether an early-exit safety-only checker would run faster. Nor does it compare the expressive power of different mature verification tools: all four subsets use the same reference semantics and exporter.

8.4 RQ5: Split delivery, shared resources, and reservation

All 72 concurrency configurations complete. Atomic mode has 24 cases and no contract failure. Reserved mode also has 24 and no failure. Split mode has 24 cases, of which 12 fail. P5 fails in those 12 contexts, and P9 fails in six shared-key contexts. Short budgets mask delivery faults in the remaining split cases. Table 14 gives all counts, retaining those short-budget controls.

Table 14. All concurrency contexts, including short budgets. Failures in split mode are intended negative controls.

Mode	Cases	Complete	Capped	Any fail	P9 fail	P5 fail
atomic	24	24	0	0	0	0
split	24	24	0	12	6	12
reserved	24	24	0	0	0	0

The separate-key controls distinguish two mechanisms. With separate candidates, one session cannot duplicate another session's candidate, but the candidate can still expire between prepare and commit. With a shared candidate, two sessions can both prepare before either commits. The second commit then observes a previously consumed candidate and produces both a stale-delivery observation and a single-use violation. This is a concrete effect of changing the atomicity boundary, not a cryptographic flaw in QKD.

Table 15. Actual shortest BFS counterexample prefix for shared-key duplicate delivery. Prefix events are retained in the JSON witness.

Step	Action
1	s0:SelectRoute(0)
2	s0:Authenticate
3	s0:Raw(0)
4	s0:EC(0)
5	s0:PA(0)
6	s0:Confirm
7	s0:Store
8	s0:PrepareDelivery
9	s1:SelectRoute(0)
10	s1:Authenticate
11	s1:Raw(0)
12	s1:EC(0)
13	s1:PA(0)
14	s1:Confirm
15	s1:Store
16	s1:PrepareDelivery
17	s0:CommitDelivery
18	s1:CommitDelivery

Table 15 is an actual shortest BFS prefix from the shared two-session split case. It contains eighteen model actions. Each session completes the required preparation, and both prepare delivery while the shared candidate is unused. Their commits then produce duplicate release. The corresponding JSON includes state identifiers, full final session records, and the violated property. The sequence is not manually invented for illustration.

Reservation prevents the competing preparation while an owner exists and rechecks expiry and consumption at commit. Valid abort releases ownership. All evaluated reserved controls pass under those assumptions. This result demonstrates a reference-model repair across the selected contexts. It does not prove that a particular key-store transaction, lease implementation, or crash-recovery mechanism implements the repair correctly.

Figure 9 presents both the state-space effect of delay and the delivery-mode failure counts. The modes differ intentionally in internal action boundaries. Their safety comparison is therefore paired by service context, while their state counts and runtime are consequences of the chosen representation. A lower time in one run is not evidence that a mode is generally more efficient.

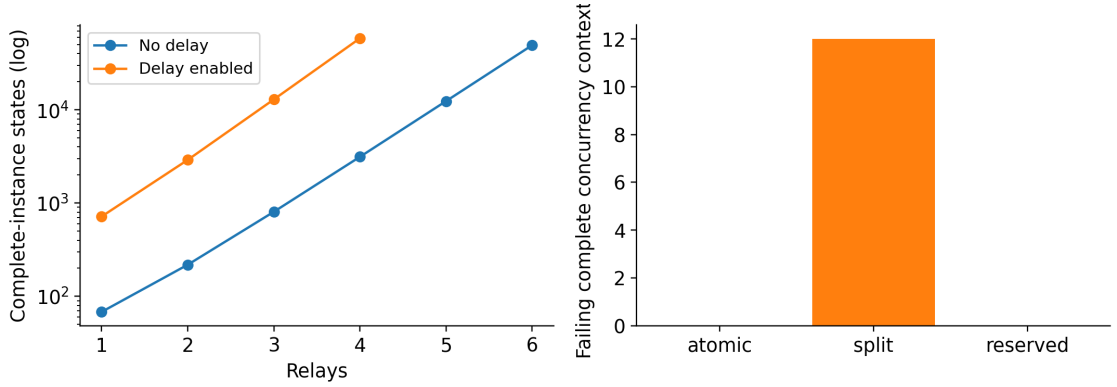


Fig. 9. Sensitivity and repair controls. Left: completed structural instances with failures enabled and slack budgets; capped delayed cases are omitted rather than plotted as zero. Right: failing complete concurrency contexts by delivery mode.

8.5 RQ6: Exhaustive verdicts and sampled testing

The thirty testing scenarios contain 24 exhaustively failing scenarios and six without a contract failure. The two schedulers, twenty seeds, and 100 episodes per seed produce 120,000 sampled executions. The audit replays 970,037 recorded actions. No episode reaches the 200-action cutoff. A terminating episode is simply one observed path; it is not a universal progress result.

At a budget of 100 traces per seed trial, uniform selection finds at least one witness in 132 of 480 failing-scenario seed trials. The progress-biased scheduler does so in 442 of 480. Pooling the twenty seeds for scenario coverage, uniform selection reaches a witness in sixteen of the 24 failing scenarios, while the progress-biased scheduler reaches all 24. Smaller budgets retain the expected monotonic prefix relation, as Table 16 shows.

Table 16. Enabled-action testing on exhaustively failing scenarios. A hit is at least one valid witness in a seed trial. Scenario coverage pools the twenty seeds.

Scheduler	Trace budget	Seed hits	Seed trials	Scenarios hit
uniform	10	93	480	7
uniform	50	122	480	10
uniform	100	132	480	16
progress_biased	10	375	480	22
progress_biased	50	424	480	23
progress_biased	100	442	480	24

The difference is understandable within these policies. Uniform enabled-action selection often takes a failure branch before a deep delivery condition is reached. Weighting failure and expiry less heavily lets more episodes reach storage and commit. This is not evidence that random testing is generally weak or that the selected weights are optimal. A coverage-guided method, targeted scenario tests, or another scheduler could behave differently.

Figure 10 shows the budget and first-witness curves. The complete graphs serve as the finite-model reference: the sampling experiment can miss a known reachable failure. Conversely, every sampled failure is confirmed against its scenario's exhaustive failing-property set. The six non-failing scenarios produce no false witness in the replay audit.

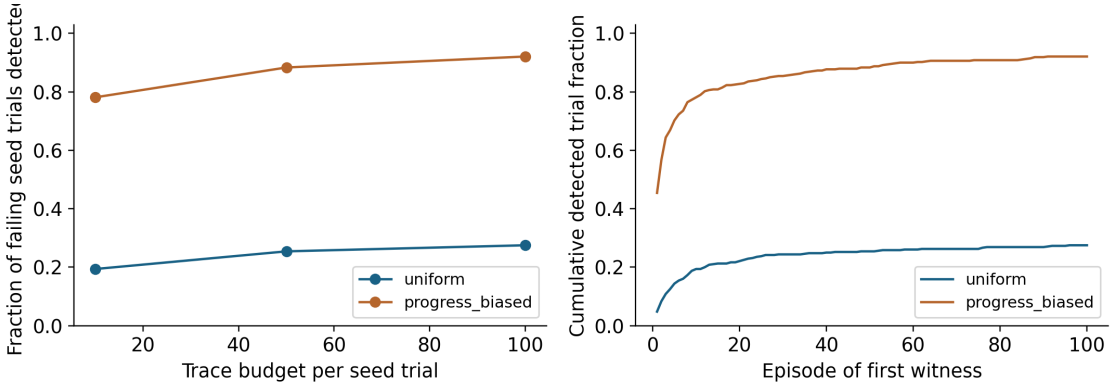


Fig. 10. Scheduler-dependent witness discovery. Left: seed-trial hit fractions at three prefix budgets. Right: cumulative first-witness fractions, retaining trials with no witness in the denominator. These are model-testing observations, not deployed-network failure probabilities.

Table 17. Selected seed-trial hit rates at 100 traces, with descriptive 95 percent Wilson intervals. Intervals concern the configured synthetic scheduler, not deployed QKD reliability.

Scenario	Scheduler	Hits	Wilson interval
skip_auth_r2	uniform	0/20	[0.000, 0.161]
skip_auth_r2	progress_biased	5/20	[0.112, 0.469]
skip_ec_r2	uniform	20/20	[0.839, 1.000]
skip_ec_r2	progress_biased	20/20	[0.839, 1.000]
wrong_session_ack_r2	uniform	0/20	[0.000, 0.161]
wrong_session_ack_r2	progress_biased	20/20	[0.839, 1.000]
expired_delivery_r2	uniform	1/20	[0.009, 0.236]
expired_delivery_r2	progress_biased	17/20	[0.640, 0.948]
missing_timeout_r2	uniform	20/20	[0.839, 1.000]
missing_timeout_r2	progress_biased	20/20	[0.839, 1.000]
retry_loop_r2	uniform	0/20	[0.000, 0.161]
retry_loop_r2	progress_biased	20/20	[0.839, 1.000]
split_shared1	uniform	3/20	[0.052, 0.360]
split_shared1	progress_biased	20/20	[0.839, 1.000]

Table 17 gives selected per-scenario hit counts and Wilson intervals over twenty seed trials. These intervals are intentionally scenario-specific. Pooling heterogeneous fault contexts into one interval would obscure that one fault may occur near the initial state and another only after a long service prefix. Even a high seed-trial hit rate remains a property of the configured scheduler and model, not a reliability estimate for real QKD equipment.

8.6 RQ7: State growth, repeated time, and memory

The dedicated performance family extends the no-delay chain experiment to seven relays. Its reachable state counts grow from 68 at one relay to 197,006 at seven. For the seven-relay setting, median core verification time is 7.922 seconds, with a retained range of 7.343–9.148 seconds. Median peak Python-worker RSS is 262.5 MiB. These observations are local measurements, not a claim about large physical-network capacity.

Table 18. Fresh-process measurements: seven measured replicates after one discarded run per setting. Time excludes graph export and C++ checking. RSS is the peak of the Python worker process.

Setting	Runs	States	Median s	Min s	Max s	RSS MiB
chain_r1	7	68	0.001	0.001	0.005	12.6
chain_r2	7	218	0.004	0.003	0.012	12.8
chain_r3	7	806	0.019	0.016	0.029	13.4
chain_r4	7	3,134	0.076	0.067	0.101	16.0
chain_r5	7	12,398	0.400	0.352	0.460	26.9
chain_r6	7	49,358	1.600	1.558	2.011	72.0
chain_r7	7	197,006	7.922	7.343	9.148	262.5
two_sessions_atomic	7	4,665	0.227	0.190	0.622	17.8
two_sessions_reserved	7	5,074	0.222	0.165	0.253	18.6
two_sessions_split	7	5,275	0.235	0.186	0.334	18.9

Table 18 reports every measured setting. The two-session atomic, split, and reserved cases have 4,665, 5,275, and 5,074 reachable states respectively. Their median times are close in this environment, while their ranges overlap. The study therefore does not claim a meaningful speed advantage for one delivery mode. The main comparison concerns the exposed interleavings and safety outcomes.

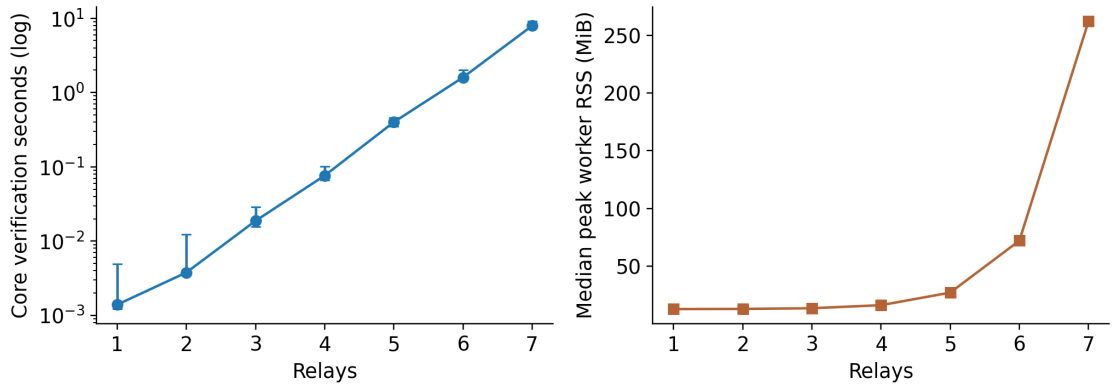


Fig. 11. Repeated performance observations for the chain family. Time bars show the minimum and maximum of seven retained runs around the median. Memory is median peak Python-worker RSS. Export, compression, and C++ checking are excluded from core time.

Figure 11 shows rapidly increasing time and memory under explicit-state composition. The baseline chain experiment and the delayed structural sweep should not be conflated: the latter can reach its 100,000-state cap even when a larger no-delay chain completes under the separate 300,000-state performance cap. Both parameters and resource policy determine which result is observed.

The measured seven-relay instance is larger than any complete grid case because it uses that higher cap and no delay actions. This is why the manuscript distinguishes the largest complete grid graph from the largest complete performance instance. Neither count is extrapolated into a national-network verification claim. More realistic queues, inventories, and request streams would change the state space substantially.

8.7 RQ8: Cross-check and replay agreement

The Python and C++ checks agree on all 513 completed grid graphs. The C++ checker obtains the same reachable counts and safety-label aggregate and agrees on the existence of a nonterminal cycle. All compressed graph hashes match their recorded digests after integrity repair. The audit also replays all 451 stored exhaustive witnesses: 407 safety prefixes and 44 progress lassos.

The safety prefixes range from one to 26 actions, with median twelve. Progress lassos contain ten to 25 recorded edges, with median seventeen. Safety prefixes are shortest under unit-action BFS distance. The progress lassos are valid but are not claimed shortest. These lengths describe the abstraction's actions, not elapsed physical time or the number of implementation instructions needed for a repair.

All 120,000 sampled traces replay, and every reported property failure belongs to the exhaustive scenario failure set. The audit records zero false witnesses and zero cutoffs. Thirteen targeted unit tests also pass. These controls support the internal consistency of the delivered dataset and the graph computations.

The agreement has a precise boundary. Both graph checking and trace replay originate from the supplied transition generator. The C++ implementation reads safety labels rather than deriving them from a second QKD model. Thus the evidence does not independently validate the domain abstraction. It supports a reproducible finite-model experiment whose assumptions and outputs can be inspected and challenged.

9 Independent Baselines and Implementation Evidence

9.1 RQ9: a separate specification and matched execution testing

Agreement between two algorithms over the same exported graph cannot validate the transition generator or its safety labels. We therefore author a separate `ResourcePool.tla` module and execute it with TLC [13]. The module directly represents waiting, prepared, delivered, aborted, free capacity, the event clock, and a persistent stale-delivery flag. It is not generated from the Python transition function and does not import a Python graph. The same authors supply both encodings, so this is software and encoding diversity, not an independent replication by another team.

The comparison covers the same 306 configurations and the conjunction of I1–I3. TLC reports 245 passing and 61 failing configurations, agreeing with Python in every case. All 245 complete passing TLC explorations also have the same reachable-state counts as Python. These are the 204 atomic/reserved controls and 41 passing split cases. The independent module, constants, commands, raw TLC logs, exit codes, and log digests are supplied. We use the official release `v1.7.4`, whose embedded checker identifies itself as TLC 2.19; the JAR is checksum pinned.

This baseline deliberately does not compare I4. The Python checker evaluates universal termination on a totalized operational graph. A TLA+ temporal specification admits stuttering and requires an explicit progress interpretation before its liveness claim can be compared. Adding fairness merely to obtain agreement would change the question. We compare aggregate safety only, without claiming a translation of the temporal semantics. TLC deadlock checking is disabled because terminal states have no enabled next action; this baseline does not establish nonterminal deadlock freedom. TLC stops at its first safety error, while the Python run exports its complete graph. Therefore failed-case state counts and their run times are not a matched performance race. No speed advantage is claimed.

For a sampling baseline, we use uniform choice over enabled labeled actions on the same 306 Python models. Ten fixed seed trials per configuration each execute 100 episodes. Every episode stops at a terminal state; the supplied structural bound prevents an unreported cutoff. This adds 306,000 executions and 3,181,452 actions, separately from the earlier 120,000-trace experiment. Of 610 seed trials on the 61 exhaustively failing cases, 180 find a safety witness within

one episode, 523 within ten, and 579 within 100. There are no false-positive trials. Pooling all ten seeds at the largest budget reaches 60 of the 61 failing cases. The missed split case has eight requests, capacity five, and both budgets twelve; exhaustive checking finds excess delivery. Repeated trials are observations of this chosen scheduler, not independent bugs or estimates of deployed failure rates.

Table 19. Matched resource-model safety checks and seeded execution testing. Random-testing hits count failing-case seed trials, not independent defects.

Method	Scope	Observed result
Python exhaustive	306 cases	245 safe; 61 unsafe.
TLC, separate TLA+ encoding	Same 306 cases	245 safe; 61 unsafe; no disagreement.
TLC complete passing cases	245 cases	Equal reachable-state counts in all 245.
Random testing, 1 executions	610 failing-case trials	180/610 hits.
Random testing, 10 executions	610 failing-case trials	523/610 hits.
Random testing, 100 executions	610 failing-case trials	579/610 hits.

Table 19 separates a complete finite-model verdict from a finite sampling hit. The baseline establishes agreement on the stated safety problem and shows what this simple testing policy misses. It does not compare against optimized concolic testing, CHESS, or Coyote.

9.2 RQ10: executing the delivery boundary in SQLite

We implement a small key-lifecycle controller in Python and SQLite. This is newly written research software, not a deployed QKD controller. It stores synthetic key identifiers, lifecycle status, reservation owners, delivery records, and operation events. It stores no secret key bits. Each request uses its own database connection. Atomic selection and consumption occur in one `BEGIN IMMEDIATE` transaction; SQLite's transaction rules provide the concrete mutual-exclusion boundary [14]. Reserved mode acquires an owner-tagged reservation transactionally and checks ownership and freshness again at commit. Split mode intentionally trusts the earlier selection without rechecking ownership or expiry. The test subject does not import the abstract model.

The design reuses the seventeen measured capacity profiles. Each profile is crossed with three delivery modes, two controlled schedules, and twenty repetitions: 2,040 batches and 11,760 request executions. Request counts are two, four, or eight. A barrier ensures that every non-atomic prepare finishes before any commit begins. In the prepare-barrier schedule, all requests use controlled clock value zero. In the expiry-boundary schedule, prepare uses zero and commit uses the expiry boundary three. These integer clock values are test inputs, not measured device times. The batches test competing database operations and the selected boundary schedules, not all thread interleavings or all SQLite failures.

Table 20 reports three implementation findings. *Duplicate* means two completed requests received the same key identifier. *Excess* means completed deliveries exceeded initial capacity. *Expired* means a recorded delivery occurred at or after its expiry boundary. All 1,360 atomic/reserved batches avoid these findings. In each split schedule, 280 of 340 batches contain duplicate identifiers and 220 exceed capacity. The expiry schedule additionally produces expired deliveries in 280 batches. Columns overlap and must not be added. The repeated barrier experiments are stability checks of chosen schedules, not 2,040 distinct software defects.

Table 20. SQLite implementation batches. Counts are batches exhibiting each finding; columns overlap. Each cell repeats seventeen profiles twenty times.

Mode	Schedule	Batches	Duplicate	Excess	Expired
atomic	prepare barrier	340	0	0	0
atomic	expiry boundary	340	0	0	0
split	prepare barrier	340	280	220	0
split	expiry boundary	340	280	220	280
reserved	prepare barrier	340	0	0	0
reserved	expiry boundary	340	0	0	0

9.3 What projection agreement does not guarantee

We also execute all eighty stored resource-model safety witnesses against the SQLite controller. The harness realizes each labeled operation through the database API, advances the controlled clock according to the model event, and compares the resulting count projection after every action. All 430 projected snapshots agree. For a database state x, the projection counts request phases and uses

$$free(x) = C - |Deliveries(x)| - |Held(x)| - |Wasted(x)|.$$

An expired aborted reservation contributes to $Wasted$, not to reusable capacity. The observed agreement validates these finite witness executions. It is not a proof that every implementation behavior refines the model.

More importantly, the projection omits concrete key identity. In each split schedule, sixty batches reuse an identifier while remaining within the capacity bound. They occur at the three positive profiles with $C = N$, each repeated twenty times. For example, with two free identifiers, both requests can select identifier zero before either consumes it. Two deliveries respect $d \le C = 2$, yet key zero has been released twice. Thus the implementation requires an additional identity contract,

$$J_1 : \quad \forall r_1 \neq r_2 \in Completed, \quad keyId(r_1) \neq keyId(r_2),$$

for this no-sharing key-unit interface. The count-based resource model does not establish J_1. Atomic selection or owner-tagged reservation prevents the observed reuse in this implementation, but a broader correctness claim would need a key-identity model and a proved relation to the storage code. This distinction is the main lesson of the implementation experiment: equal capacity projections can hide an allocation error.

Figure 12 combines the implementation and sampling results without equating their denominators. SQL histories, request-level outcomes, witness projections, seeds, and verification commands are included. Eight additional controller tests cover ownership, reuse without excess capacity, expiry, abort release, and zero-capacity behavior; the complete suite contains twenty-eight tests.

10 Software Engineering Implications

10.1 Contracts should follow event boundaries

The delivery experiment shows why event boundaries matter. An implementation can be correct under an atomic abstract action but fail when that action is split. Moving a precondition check earlier does not preserve its truth until use. The model therefore measures freshness at the actual commit. An earlier snapshot remains useful diagnostic information, but is not accepted as proof that the later event was authorized.

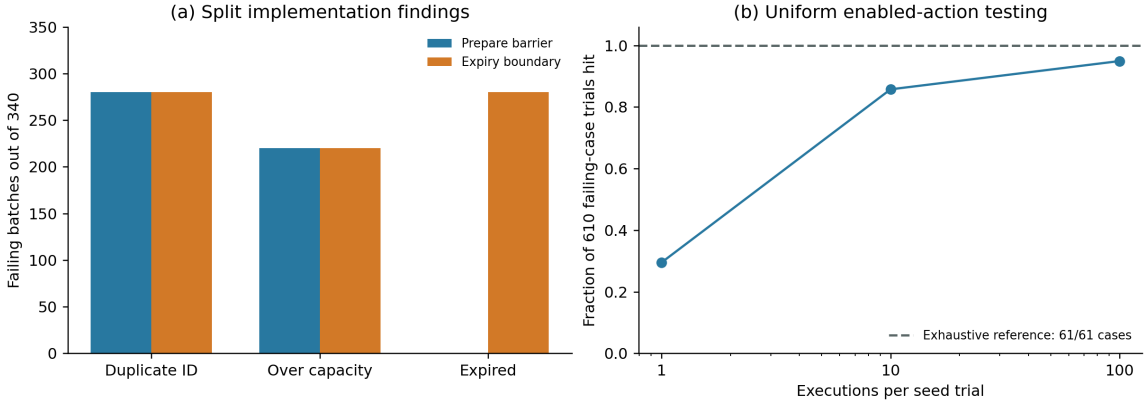


Fig. 12. Implementation findings and matched sampling evidence. The left panel counts findings in split batches; atomic and reserved counts are zero. The right panel uses 610 failing-case seed trials, with exhaustive reference verdicts shown only as a detection ceiling.

A persistent update is a separate concern. Even when a delivery receipt is valid, omitting the consumed update can allow reuse later. P9 checks that state change independently of P5. Likewise, an execution can satisfy all delivered-key implications by never delivering. P8 and explicit successful-delivery reachability observations prevent these outcomes from being conflated with usable service correctness.

The practical lesson is to link each requirement to the point where its evidence is valid. Authentication and confirmation reports concern the current session. Freshness concerns actual release. Consumption concerns the state after release. An abort cause concerns the moment of termination. A retry rule concerns an entire possible future execution. A single broad assertion at one endpoint cannot directly observe all of these obligations.

10.2 Model repairs require implementation arguments

Reservation is an effective repair in the reference modes because ownership acquisition is atomic and commit revalidates the candidate. In a real system, those requirements must be implemented with a justified transaction, lock, compare-and-swap operation, or equivalent mechanism. The present model does not select a database isolation level or prove a lock implementation. It provides a precise target for such an argument.

The split counterexample can guide an integration test: pause one request after checking a candidate, allow a competing request to reach the same point, then resume both commits. A second test can pause after prepare and force expiry before commit. Those are actionable implementation scenarios. The SQLite study executes these boundary scenarios in newly implemented storage code. It does not validate an external production controller.

Table 21 separates the reference obligation from deployment evidence. The SQLite experiment supplies bounded evidence for the local selection, reservation, and commit boundary. The broader deployment obligations in the table remain unproved; an abstract graph pass alone completes none of them. This distinction is essential for software engineering use: a model can improve design reasoning without automatically establishing that production code conforms to it.

Table 21. Reference obligations and implementation evidence that would be needed for deployment assurance.

Responsibility	Reference obligation	Additional implementation evidence
Session handler	Session-bound authentication and acknowledgement	Message parsing, identity validation, replay handling.
Post-processing adapter	EC precedes PA completion	Correct mapping of device reports and failures.
Endpoint interface	Confirmation before actual delivery	Agreement on actual key identifier and session.
Key-store commit	Freshness at use and consumed update	Transactional or synchronization argument.
Reservation handling	Exclusive owner and release on abort	Lock/lease correctness and crash cleanup.
Timeout/retry logic	Valid abort and termination	Scheduler, real-time, and recovery assumptions.

10.3 Dataset scale and evidence quality

The experiment is larger in computational coverage than a single topology demonstration. However, increasing the number of configurations does not make the model more realistic by itself. Many cases vary budgets around the same small service abstraction. The dataset is useful because it exposes those choices and preserves negative and unknown outcomes, not because every row is a new independent system.

Similarly, many sampled traces do not replace exhaustive evidence. The scheduler comparison shows what two selected testing policies encounter under a finite budget. The matched resource testing baseline also uses a simple enabled-action scheduler. Neither comparison evaluates human testing teams, coverage-guided fuzzers, or production monitoring. The graph cross-check adds a different algorithmic implementation but shares the exported labels. The complete argument therefore combines several limited controls rather than presenting one large number as decisive evidence.

11 Related Work

Table 22 summarizes the scope and evidence of the representative sources discussed below.

QKD network key management spans architecture, interfaces, and lifecycle decisions. Dervisevic et al. [1] review this area. ETSI GS QKD 014 defines an application-facing delivery interface [2]. ITU-T Y.3804 and Y.3830 address control/management and key-supply resilience [3, 7]. These sources motivate service responsibilities here. The prototype is not a standards-conformance checker, and the contracts are not presented as an exhaustive encoding of every normative clause.

Formal verification of quantum protocols is well established. Nagarajan et al. [15] analyze BB84 using probabilistic model checking. Gay et al. [16] study automated checking of other quantum protocols. These works reason about quantum-protocol behavior, while the present model treats primitive completion as an abstract report. Applying model checking to a QKD-related subject is therefore not a novelty claim.

Grigolo [17] studies formal security verification of the ETSI delivery protocol using Tamarin, according to the public repository abstract. This is close to the application boundary examined here. The distinction is in the object and evidence: the present study examines coordination lifecycle obligations, a split check/commit race, reservation, and parameterized synthetic model contexts. It does not compare security guarantees or claim superiority over that symbolic analysis. The dissertation's full text is restricted, so the discussion is limited to its public abstract.

Ibrahim et al. [18] describe an implemented QKD-integrated SDN key-management architecture and lifecycle state machine. Their work includes operational implementation and measurements, while machine-checked symbolic verification is identified as future work. The present study provides executable abstract-model evidence but lacks that

implementation connection. Its state counts and seeded trace hit rates are not operational performance measurements comparable to their testbed results.

For software verification generally, temporal model checking and formal specification provide the underlying methods [4, 19]. Industrial distributed-service experience motivates design-level reasoning [5]. Mutation analysis supports sensitivity studies of selected faults [11]. The present paper uses conventional techniques in an explicit coordination benchmark. It introduces neither a new fixed-point algorithm nor a general automated program-verification method.

Model-to-code correspondence has stronger precedents than finite trace replay. IronFleet combines state-machine refinement with implementation verification [20]; Verdi provides a Coq framework for verified distributed implementations [21]. P brings executable event-driven code and model checking into one language [22]. More recently, Ouyang et al. use multi-grained TLA+ specifications and code-conformance checks for ZooKeeper [23]. These studies address implementation correctness at a broader scale or with stronger proof obligations. The present SQLite experiment establishes observed projection agreement for selected histories, with an explicit key-identity omission. It claims neither their proof strength nor a replacement for their frameworks.

The delivery boundary is also related to established concurrent-object semantics. Linearizability specifies when concurrent operations can be understood as atomic abstract operations [24]. Line-Up checks deterministic linearizability [25]. Our immediate transactions supply a concrete local serialization boundary, but we do not run a general linearizability checker or prove a refinement theorem. The duplicate-identifier observation is narrower: capacity accounting alone cannot establish the no-sharing identity contract of this interface. Elle studies anomalies inferred from transactional observations [26]; our SQL event logs serve a much smaller, application-specific oracle and do not test SQLite isolation itself.

Systematic concurrency testing has long controlled schedules to expose rare interleavings. CHESS systematically explores concurrent executions [27], while Coyote offers controlled testing of asynchronous code [28]. Our prepare barrier follows that general testing principle but exercises only two selected schedules and does not implement either tool's search algorithms. DART combines random testing with path-directed analysis [29]; our uniform enabled-action baseline has no constraint solving or feedback direction. Its missed cases therefore do not demonstrate superiority over those established tools. The model-based testing taxonomy of Utting et al. [30] helps locate the witness harness: model-derived tests are applied to separate executable code with a finite observable projection.

TLC is an established explicit-state checker for TLA+ [13]. Executing a separately written module removes the dependence on Python-exported transition edges and labels for aggregate resource safety. It does not remove shared requirements assumptions. This is the specific role of the baseline here, rather than a claim to a new checking algorithm or a benchmark ranking.

Benchmark design also has an established literature. The PRISM benchmark suite groups models, parameters, properties, and expected results to support systematic tool experiments [9]. VeriQKD-Bench follows the inspectability goal but is a distinct, much narrower synthetic artifact. It does not supply a tool leaderboard or use a PRISM implementation as its oracle. The contribution is the specific state abstraction, delivery-boundary comparison, contract decomposition, and accompanying auditable data. The public IrelandQCI measurements add empirical operating conditions through an explicit supply mapping; they do not supply controller fault ground truth.

Table 22. Scope comparison with representative sources. Absence of a head-to-head experiment is explicit.

Source	Main evidence	Relation to this study
QKD survey [1]	Key-management synthesis	Domain context; no performance comparison here.
BB84 checking [15]	Quantum-protocol analysis	Different abstraction and security question.
Delivery analysis [17]	Symbolic delivery-protocol analysis	Closely related interface; no superiority claim.
QKD-SDN KMS [18]	Implementation and testbed results	Stronger implementation connection; different metrics.
PRISM benchmark [9]	Models and property benchmarks	Benchmark practice; not an executed baseline tool.
TLC [13]	Explicit-state checking of a separate specification	Executed safety baseline on all 306 resource cases.
IronFleet/Verdi [20, 21]	Implementation-oriented formal proofs	Stronger implementation guarantees than selected replay.
P and multi-grained specifications [22, 23]	Executable models or code-conformance checks	Related model/code connection; different systems and scope.
CHESS/Coyote [27, 28]	Controlled concurrency testing	Related scheduling principle; not executed baseline tools.
Linearizability and Line-Up [24, 25]	Concurrent-object correctness and checking	Our local transactions are not a general proof.
This study	Public hardware data and finite-model checks	Measured supply mapping; controlled fault, race, and repair experiments.

12 Threats to Validity

Construct validity. The model represents completion reports rather than cryptographic operations. Confirmation is a Boolean report, not a comparison of real key bits. Agreement is an acknowledgement mask plus an identity-validity flag, not a complete message protocol. Expiry is nondeterministic. Budgets count local events rather than physical time. Consequently the injected model failures are failures of the defined coordination contracts, not demonstrations of cryptographic compromise. The hardware measurements are analyzed separately from those failures.

Internal validity. The transition generator, safety predicates, export, and replay share Python source semantics. The C++ checker independently implements reachability and a different progress algorithm but consumes the same graph and labels. The TLC safety baseline instead explores a separately written specification. Both specifications are authored in this study and may share mistaken requirements. Witness projection checks exercise only their eighty chosen database histories and omit key identity. Agreement can therefore coexist with a modeling or labeling mistake. Unit tests, boundary tests, manual invariant arguments, replay, and checksums reduce specific risks; they do not constitute a mechanically verified checking toolchain.

Fault-model validity. Faults were hand-authored with knowledge of the contracts. Their detection rates can be optimistic for that reason. Multiple parameter contexts of one operator are not independent fault discoveries. Context masking is retained in denominators, while only the explicitly checked diagnostic-label control is treated as equivalent under the state-observation boundary. Higher-order results cover selected pairs, not arbitrary combinations or real defect clusters.

External validity. The coordination benchmark is synthetic, while the measurement extension uses public hardware key-rate and QBER records. The SQLite controller is new research code with actual local database transactions, but no external controller source, production incident report, or operational message trace is verified. The original benchmark has at most three sessions; the separate resource abstraction has up to eight identical requests. Route alternatives

in the original model are hop counts. There are no general packet queues, unbounded arrivals, key replenishment, compromised relays, crash-recovery persistence semantics, or cryptographic adversaries. The SQLite repair uses immediate transactions and owner checks. It does not establish crash recovery, distributed transaction correctness, or complete refinement. Controlled clocks and barriers expose selected schedules; they do not provide a wall-clock or scheduler probability model.

Testing validity. Random execution testing is conditioned on two chosen enabled-action schedulers and fixed scenario configurations. The weights are design choices rather than calibrated probabilities. A failure to sample a witness is not a pass. Seed-trial intervals describe this sampling experiment, not the distribution of real network failures. The random-testing comparison is with exhaustive reachability, not optimized testing tools or experts. The TLC comparison covers only aggregate resource safety; it does not establish equivalent liveness semantics.

Performance validity. Seven measured repetitions provide a limited view of process variability in one environment. RSS includes Python runtime and representation overhead. Core timings exclude export and cross-check costs and should not be interpreted as end-to-end pipeline latency. No claim is made that this prototype is faster or more memory efficient than established tools. TLC process timings include JVM startup and may stop at a first error, so they are not compared as speedups against complete Python graph export. Capped cases are unknown, and their counts make the resource boundary visible.

Conclusion validity. Configuration-counted state totals can overstate diversity when related cases share graph structures. The paper therefore also reports exact exported graph-serialization counts. Those counts are not graph-isomorphism classes. The study uses descriptive comparisons and does not claim statistical generalization to a population of systems. A larger synthetic dataset alone does not establish generality beyond the modeled domain.

13 Conclusion

This study provides an executable coordination model, an expanded synthetic benchmark, and a measurement-driven resource study using public QKD hardware data. It separates delivery-time authorization, post-delivery updates, intermediate ordering, abort validity, and progress. The split delivery experiment exposes stale checks and shared-key races. Reservation supplies a model-level repair under explicit atomicity assumptions.

The dataset includes 522 canonical grid configurations, of which 513 complete and 9 remain unknown at their caps. Complete graphs agree across two graph-checking implementations. Additional experiments provide 120,000 sampled executions, repeated resource measurements, and replay audits. The evidence demonstrates contract sensitivity and parameter effects within the reference semantics. Separately, 48,507 primary published measurements produce 582,084 traceable capacity mappings and seventeen observed saturated capacity classes. All 306 resource-model cases complete. Atomic and reserved modes pass all 204 controls, while split mode fails in 61 of 102 cases under the selected controller budgets.

The separate TLA+ encoding agrees on all 306 resource safety verdicts and all 245 complete passing state counts. An additional 306,000 random executions still miss one failing case across all selected seeds. The SQLite controller completes 2,040 batches and reproduces eighty resource witnesses with 430 matching projected snapshots. Its identifier-level observations expose reuse without excess capacity, a property outside that count projection. Atomic and reserved modes avoid the selected implementation findings, under local transactional assumptions.

The next step is to validate an identity-aware model against an independently developed controller and extend the relation to retries, persistent recovery, and distributed messages. The current evidence combines real published

operating records, bounded abstract checking, and a local implementation study. It is not deployed-controller assurance or a mechanically proved refinement.

Data and Code Availability

The supplementary artifact contains original measurements, normalized records, capacity mappings, synthetic configurations, graphs, witnesses, source code, independent TLC inputs and logs, SQL histories, testing seeds, and audit scripts. Unmodified IrelandQCI v2 source files and pinned metadata are credited to Horgan et al. and the IrelandQCI/CONNECT teams under CC BY 4.0 [6]. The newly supplied experimental code has a scoped MIT license. Third-party tools retain their original licenses.

The public repository is https://github.com/Shubha-fm/qnet-verival/tree/main/research. The original experiment archive is pinned to commit `0dcc7877755e62a216fffb95e6bf7abedc151a2f`. Its three archive parts are verified by SHA-256 before extraction. The independent-baseline and SQLite extension is pinned to commit `1549342624d2c9eaa1f0c5b4061d52aa2949ea57`. It supplies the SQLite implementation, independent baseline, new raw results, and reproduction commands. The web application elsewhere in the repository is a separate prototype, whose API outputs are not the archived experimental evidence.

References

[1] E. Dervisevic, A. Tankovic, E. Fazel, R. Kompella, P. Fazio, M. Voznak, and M. Mehic. 2025. Quantum Key Distribution Networks—Key Management: A Survey. *ACM Computing Surveys* 57, 10. https://doi.org/10.1145/3730575.

[2] ETSI. 2019. *Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API.* ETSI GS QKD 014 V1.1.1. https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01.01.01_60/gs_qkd014v010101p.pdf.

[3] ITU-T. 2025. *Quantum key distribution networks—Control and management.* Recommendation Y.3804, April 2025. https://www.itu.int/rec/T-REC-Y.3804/en.

[4] L. Lamport. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.* Addison-Wesley. https://lamport.org/tla/book.html.

[5] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. 2015. How Amazon Web Services Uses Formal Methods. *Communications of the ACM* 58, 4, 66–73. https://doi.org/10.1145/2699417.

[6] J. Horgan et al. 2026. *QKD Network Characterisation and Coexistence Datasets with Standardised Performance Metrics and Experimental Methods*, Version v2. Zenodo. https://doi.org/10.5281/zenodo.21792844. Source data licensed CC BY 4.0.

[7] ITU-T. 2025. *Quantum key distribution networks—Procedures for key supply protection and recovery for resilience.* Recommendation Y.3830, August 2025. https://www.itu.int/rec/T-REC-Y.3830/en.

[8] MITRE. 2026. *CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition.* Accessed 15 October 2026. https://cwe.mitre.org/data/definitions/367.html.

[9] M. Kwiatkowska, G. Norman, and D. Parker. 2012. The PRISM Benchmark Suite. In *Proceedings of the 9th International Conference on Quantitative Evaluation of Systems (QEST).* https://www.prismmodelchecker.org/papers/qest12.pdf.

[10] M. D. Wilkinson et al. 2016. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* 3, 160018. https://doi.org/10.1038/sdata.2016.18.

[11] Y. Jia and M. Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5, 649–678. https://doi.org/10.1109/TSE.2010.62.

[12] NIST/SEMATECH. *e-Handbook of Statistical Methods*, Section 7.2.4.1, Confidence intervals. Wilson interval discussion. Accessed 15 October 2026. https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm.

[13] Y. Yu, P. Manolios, and L. Lamport. 1999. Model Checking TLA+ Specifications. In *CHARME*, LNCS 1703. https://doi.org/10.1007/3-540-48153-2_6.

[14] SQLite Project. 2026. *Transaction.* SQLite documentation, accessed 15 October 2026. https://www.sqlite.org/lang_transaction.html.

[15] R. Nagarajan, N. Papanikolaou, G. Bowen, and S. Gay. 2005. An Automated Analysis of the Security of Quantum Key Distribution. Preprint, arXiv:cs/0502048. https://arxiv.org/abs/cs/0502048.

[16] S. Gay, R. Nagarajan, and N. Papanikolaou. 2005. Probabilistic Model-Checking of Quantum Protocols. Preprint, arXiv:quant-ph/0504007. https://arxiv.org/abs/quant-ph/0504007.

[17] G. Grigolo. 2025. *Formal security verification of key delivery protocols in quantum key distribution networks: An overview of the security and soundness of the ETSI GS QKD 014 key delivery protocol standard.* Bachelor's dissertation, University of Malta. Related-work description based on the public repository abstract. https://www.um.edu.mt/library/oar/handle/123456789/140025.

[18] R. Ibrahim, I. Khider, S. Edam, and M. Burai. 2026. Centralised Key Management System Architecture for Topology-Aware and Quantum-Resilient Security in QKD-Integrated SDN Control Planes. *IET Quantum Communication* 7, 1, e70035. https://doi.org/10.1049/qtc2.70035.
[19] E. M. Clarke, E. A. Emerson, and A. P. Sistla. 1986. Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications. *ACM Transactions on Programming Languages and Systems* 8, 2, 244–263. https://doi.org/10.1145/5397.5399.
[20] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, J. Stephenson, S. Setty, and B. Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *SOSP*. https://www.microsoft.com/en-us/research/publication/ironfleet-proving-practical-distributed-systems-correct/.
[21] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson. 2015. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *PLDI*. https://doi.org/10.1145/2737924.2737958.
[22] A. Desai, V. Gupta, E. Jackson, S. Qadeer, S. Rajamani, and D. Zufferey. 2013. P: Safe Asynchronous Event-Driven Programming. In *PLDI*. https://ankushdesai.github.io/assets/papers/p.pdf.
[23] L. Ouyang, X. Sun, R. Tang, Y. Huang, M. Jivrajani, X. Ma, and T. Xu. 2025. Multi-Grained Specifications for Distributed System Model Checking and Verification. In *EuroSys*. https://doi.org/10.1145/3689031.3696069.
[24] M. P. Herlihy and J. M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3, 463–492. https://doi.org/10.1145/78969.78972.
[25] S. Burckhardt, C. Dern, M. Musuvathi, and R. Tan. 2010. Line-Up: A Complete and Automatic Linearizability Checker. In *PLDI*. https://www.microsoft.com/en-us/research/publication/line-up-a-complete-and-automatic-linearizability-checker/.
[26] K. Kingsbury and P. Alvaro. 2020. Elle: Inferring Isolation Anomalies from Experimental Observations. *Proceedings of the VLDB Endowment* 14, 3, 268–280. https://www.vldb.org/pvldb/vol14/p268-alvaro.pdf.
[27] M. Musuvathi, S. Qadeer, T. Ball, G. Basler, P. A. Nainar, and I. Neamtiu. 2008. Finding and Reproducing Heisenbugs in Concurrent Programs. In *OSDI*. https://www.usenix.org/conference/osdi-08/finding-and-reproducing-heisenbugs-concurrent-programs.
[28] Microsoft Research. 2026. *Coyote*. Project documentation, accessed 15 October 2026. https://www.microsoft.com/en-us/research/project/coyote/.
[29] P. Godefroid, N. Klarlund, and K. Sen. 2005. DART: Directed Automated Random Testing. In *PLDI*, 213–223. https://doi.org/10.1145/1065010.1065036.
[30] M. Utting, A. Pretschner, and B. Legeard. 2012. A Taxonomy of Model-Based Testing Approaches. *Software Testing, Verification and Reliability* 22, 5, 297–312. https://doi.org/10.1002/stvr.456.

A Artifact Structure and Reproduction

The package root contains `src`, `tests`, `dataset`, `measurement`, `results`, `figures`, and `manuscript`. The measurement directory preserves the original licensed source files separately from normalized records and generated resource-model results. The dataset card documents the synthetic origin, units, fields, exclusions, and reuse limits. The main grid design is `dataset/design.json`. Input and outcome files share the case ID. Every completed grid case has a compressed portable graph. Capped cases deliberately lack a completed graph and retain null verdicts.

To obtain the complete package from a repository checkout, run the following commands from the repository root. The restoration script checks all three archive parts together against the pinned SHA-256 and refuses to overwrite an existing extraction.

```
python3 research/restore_package.py
cd research/replication/VeriQKD_Net_Measurement_Driven
```

The full pipeline requires Python 3.10 or newer and a C++17 compiler. Model generation, checking, replay, and tests use Python's standard library. Figure generation additionally requires Matplotlib and NumPy. The final validation command checks the supplied schema rules using the standard library. The delivered manuscript includes the rendered figures, so a reader compiling LaTeX does not need to regenerate them first.

```
python3 -m unittest discover -s tests -v
python3 src/run_benchmark.py
python3 src/random_testing.py
python3 src/performance.py
python3 src/audit.py
python3 src/analyze.py
python3 src/validate_package.py
```

The measurement arm is reproduced separately:

```
python3 src/acquire_measurements.py
python3 src/measurement_audit.py
python3 src/measurement_experiments.py
python3 src/analyze_measurements.py
python3 src/audit_measurements.py
```

The new implementation and baseline experiments have separate drivers:

```
python3 src/fetch_tlc.py
python3 src/run_tlc_baseline.py
python3 src/run_pool_testing.py
python3 src/run_implementation.py
python3 src/analyze_extension.py
python3 src/audit_extension.py
```

The TLC baseline requires Java. Its acquisition script pins the official JAR checksum. The run script isolates each process's temporary module directory and stores the complete console log, constants, exit code, and result. The SQLite study uses the database library bundled with Python; its recorded SQLite version is 3.53.1. SQL histories and projected witness snapshots are retained without storing secret key material. The extension adds three hundred and six thousand model executions and does not replace the earlier synthetic trace records.

The acquisition step uses the supplied files when their checksums match; it accesses Zenodo only for missing or invalid files.

These commands must be run from the extracted package directory. The benchmark controller compiles its C++ checker. It can resume existing outcome files; the reproduction guide explains how to select a fresh output directory when regenerating from scratch. Existing reference measurements should be preserved before rerunning. The script records unknown outcomes if a configured cap is reached.

For a graph-level check alone, decompress one graph into a temporary text file, compile `checker.cpp`, and pass that file to the executable. Its JSON output reports reachable vertices, edges, a violation mask, and whether a nonterminal cycle was found. This portable check does not require reconstructing the service model. It also does not reconstruct the meanings of the supplied labels.

The publication tables and figures are generated from recorded outcomes. `src/analyze.py` writes the CSV summary, aggregate JSON, LaTeX tables, numeric macros, and five evidence PNG files. Thus the numbers in the manuscript are tied to the delivered records. The primary `main.tex` entry point now uses the official ACM class in anonymous manuscript review mode. It is compiled with Libertine, Inconsolata, and newtx math fonts. The public-link variant is `public.tex`. A portable article entry point remains available as `portable.tex`.

B Metrics and Calculation Rules

In the original coordination benchmark, a complete-case pass means all nine P1–P9 properties are true. The resource arm uses its separate four I1–I4 contracts. In either arm, a complete-case failure means at least one applicable property is false. A capped case contributes to neither pass nor fail counts. For fault-context detection, the denominator includes every completed operational-variant context in that analysis. The diagnostic-label control is separately identified. Contextual survivors remain in the denominator and are not silently reclassified as equivalent.

Let D be the selected complete fault contexts and P a subset of contracts. The reported subset count is

$$Detected(P, D) = |\{c \in D : \exists p \in P,\ verdict(c, p) = false\}|.$$

This is an observed finite-set count. It is not a probability estimate over real software defects. Property overlap means the same context can fail multiple contracts. Consequently sums of per-property failure counts can exceed the number of failing contexts.

For a random-testing scenario, one seed trial contains 100 episodes under a fixed scheduler. Its prefix hit at budget b is

$$Hit_b = \mathbf{1}\{\text{at least one valid witness in episodes } 1, \ldots, b\}.$$

A missing witness does not negate the exhaustive reference failure. The CDF plots retain non-detecting trials in the denominator. They therefore do not condition on eventual detection, which would otherwise make the curves look more favorable.

For selected scenario/scheduler pairs, the Wilson interval uses $n = 20$ seed trials, k hits, and $z = 1.95996398454$. With $\hat{p} = k/n$, its center and half-width are

$$\frac{\hat{p} + z^2/(2n)}{1 + z^2/n}, \qquad \frac{z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}.$$

The bounds are clipped to $[0, 1]$ for reporting. The interval is descriptive for the synthetic sampling experiment and has no interpretation as a QKD security bound.

Configuration-counted state totals sum reachable vertices across complete cases. The same graph can contribute more than once under different case IDs. Exact graph-serialization diversity counts distinct compressed graph digests, including supplied labels and discovery order. It is not an isomorphism calculation. The paper reports both quantities to avoid equating total computation with unique behavioral structure.

Timing medians use seven retained fresh-process runs per setting. The first run is discarded. Minimum and maximum summarize the retained range; they are not confidence limits. Peak RSS is a process high-water mark, not a direct measurement of abstract graph bytes. No export, compression, or C++ cost is included in the core timing statistic.

C Evidence Boundaries

Table 23. Claims supported by the delivered evidence and claims requiring further work.

Supported within this artifact	Not established by this artifact
Exhaustive verdicts on complete finite graphs	Correctness for arbitrary network size.
Counterexamples and cycles in reference semantics	A real attack or production software vulnerability.
Graph-algorithm and separate TLC safety agreement	Independent validity of the QKD abstraction.
Selected model witnesses and SQLite reservation tests	Complete refinement or correctness of every storage history.
Seeded scheduler-dependent testing hit rates	Calibrated deployment reliability or cryptographic risk.
Repeated local process measurements	Comparative performance superiority over established tools.
Public repository, measured source data, and reproducible generators	An archival DOI for this replication package or an independent artifact badge.

Table 23 separates finite-model evidence from deployment claims. Public hardware records inform supply assumptions; they do not establish controller conformance or field failure rates.